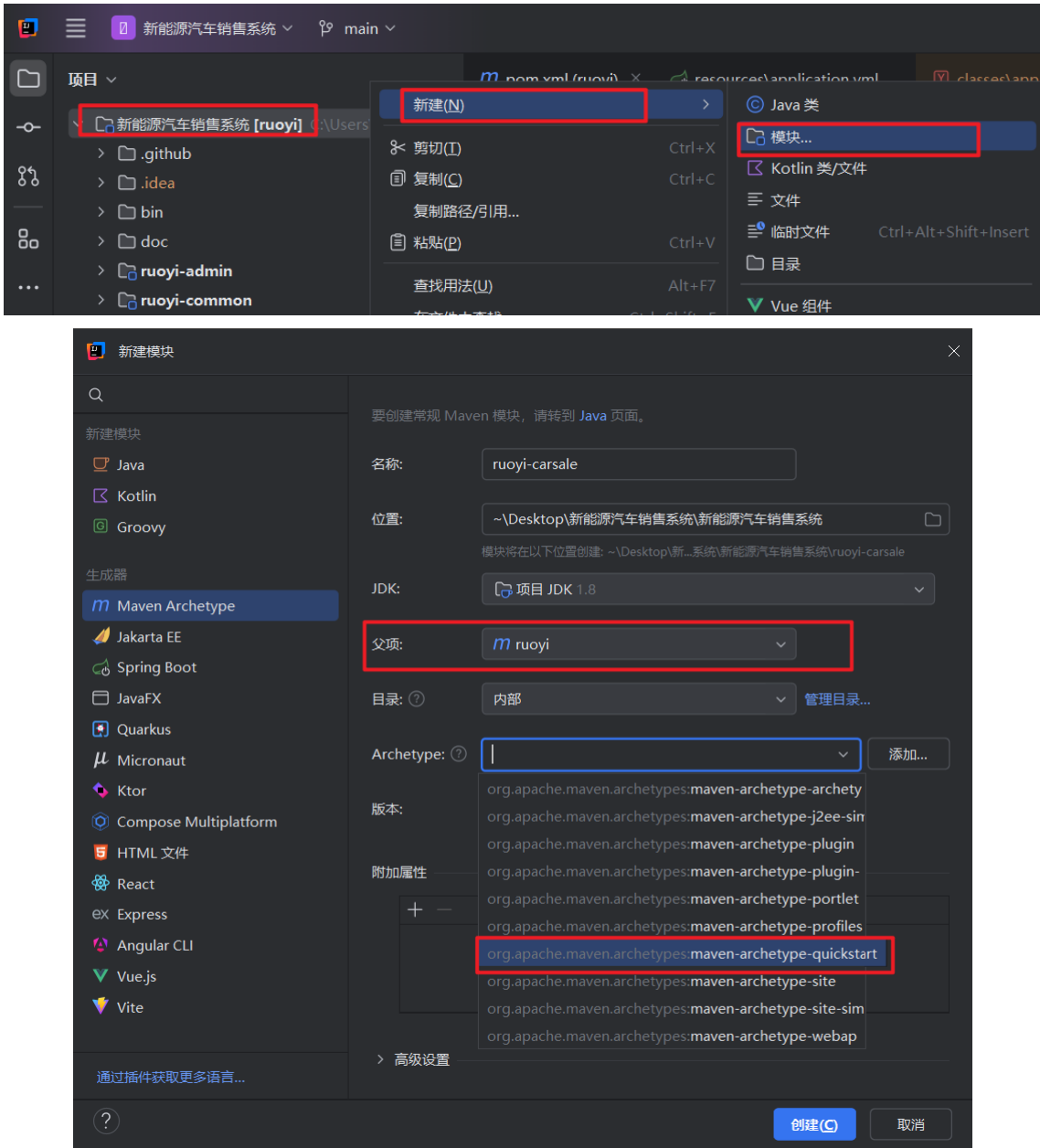
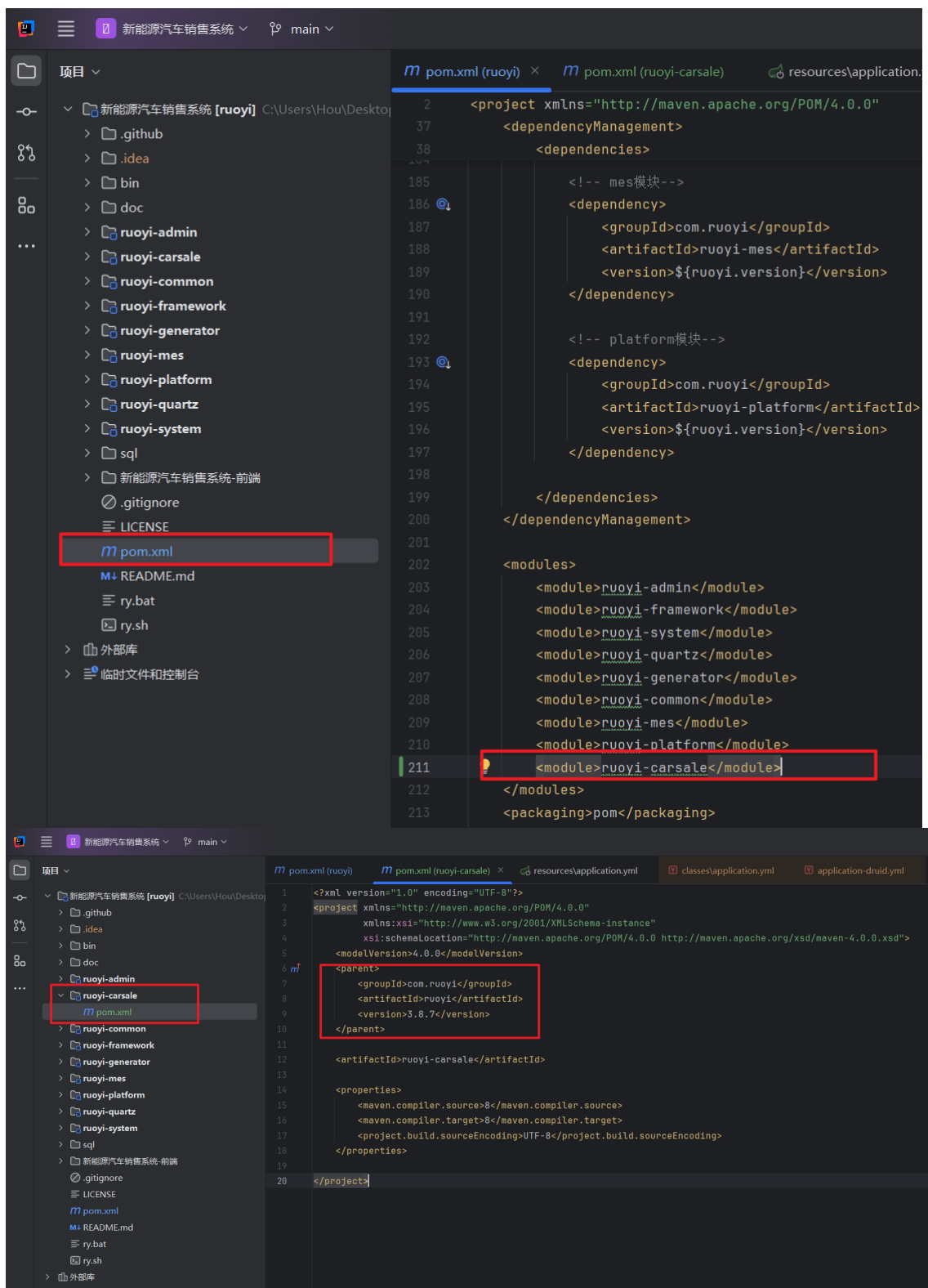


后端结构

- 1. 在项目路径下新建模块，路径要注意，要让新module继承父工程的。其实这个项目父工程和各个子模块是相互依赖的。
父工程依赖各个子模块 ---> 可以便于整个项目的打包，否则还要考虑前置模块和后续模块
各个子模块依赖父工程 ---> 便于分模块开发，好处显而易见
总体来说，建立新的业务模块，可以参考system模块【mes、platform模块也是以前做项目建立的业务模块，也可以参考】



- 2. 创建新模块之后，可以看到父工程 pom.xml 配置文件中自动添加了新模块的依赖，而新模块的 pom.xml配置文件中，同样也加入了对父工程的依赖



3. 模块新建好之后，要明确的是，后端 service、mapper 留在新模块中，controller 层统一是放到 ruoyi-admin 模块中的，Controller 要调用 Service ---> ruoyi-admin 要依赖 ruoyi-carsale，而对于这个若依项目，他还包含一些组件、框架等，所以整个项目的依赖关系是

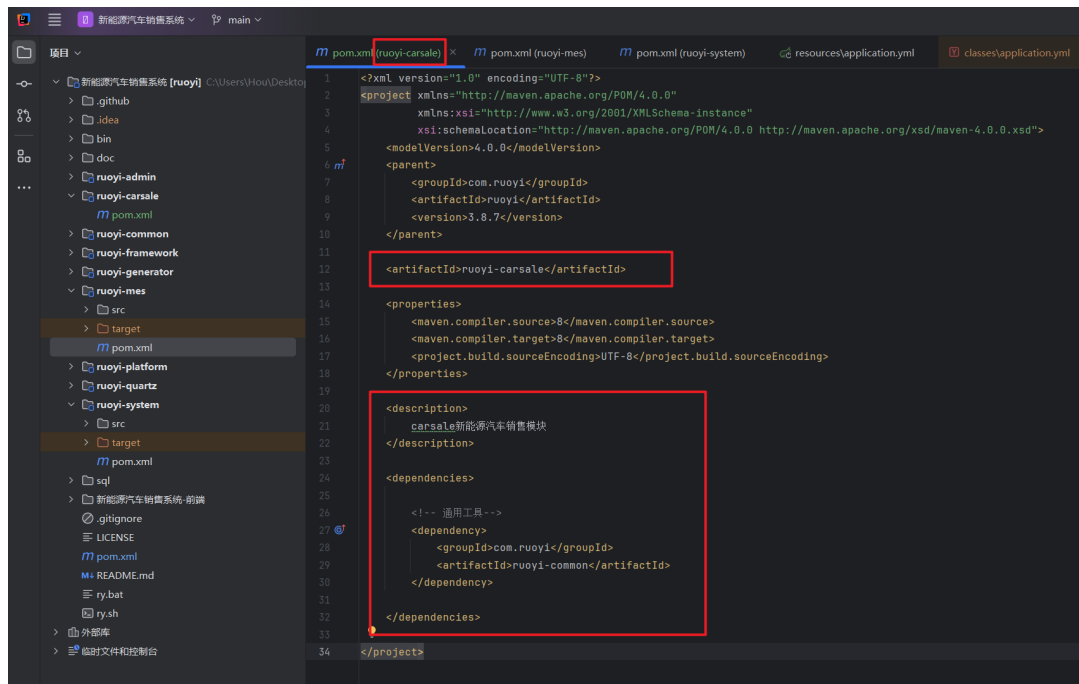
ruoyi-admin 依赖 ruoyi-framework，ruoyi-framework 依赖 各个业务模块，各个业务模块 依赖 ruoyi-common

即：admin <--- framework <--- 业务模块 <--- common

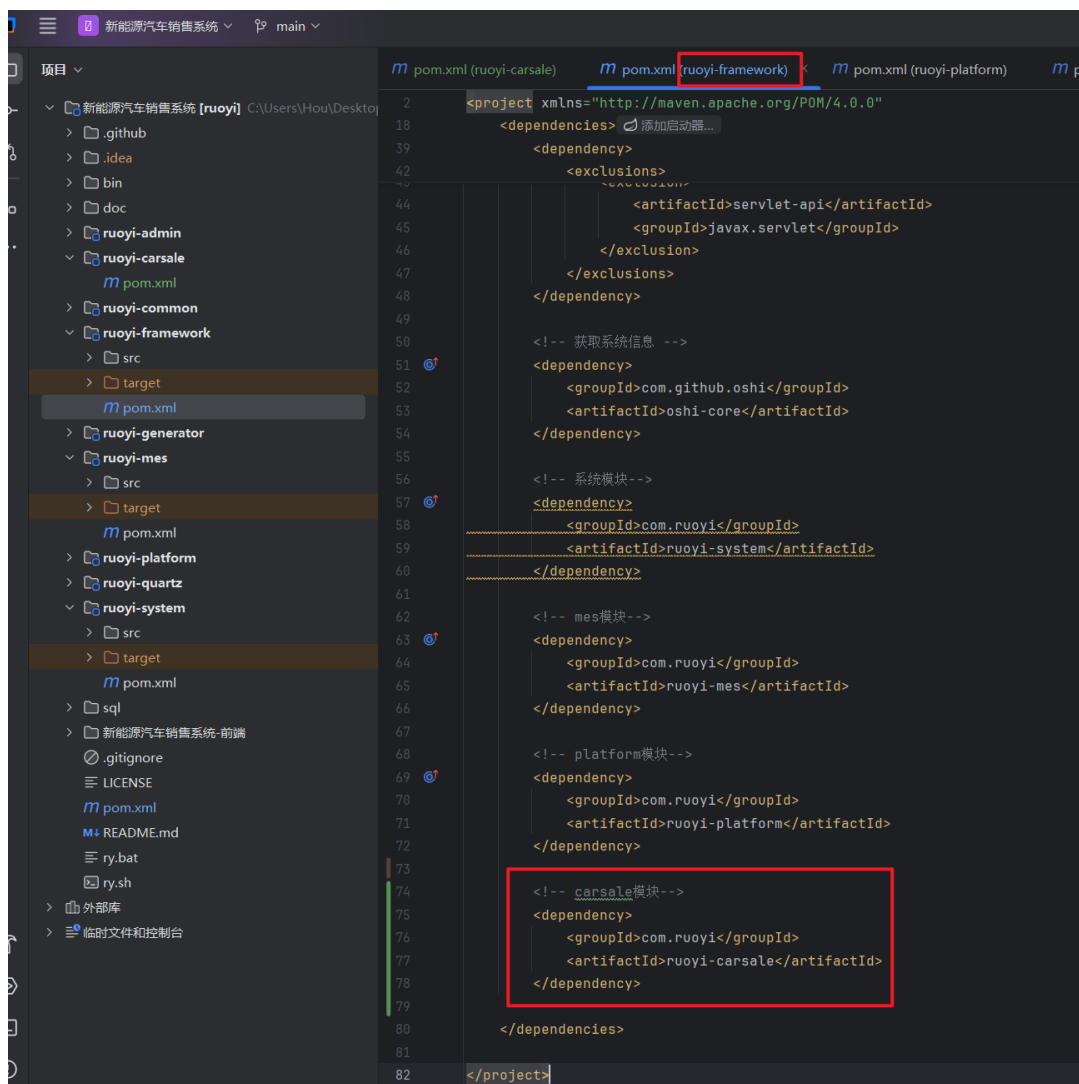
而当前创建的是业务模块 carsale，因此接下来的操作就是把 ruoyi-framework 和 ruoyi-common 模块依赖加上【注意，是业务模块加入 framework 配置文件，而 common 模块加入业务模块的配置文件中】【可以抄一抄 ruoyi-mes、ruoyi-platform 模块，这也是以前练习时候建立的一个业务模块】

最后，直接在父工程文件中声明各个依赖的版本，这样就不用再其他地方声明了，比如在 framework 中就没有声明版本

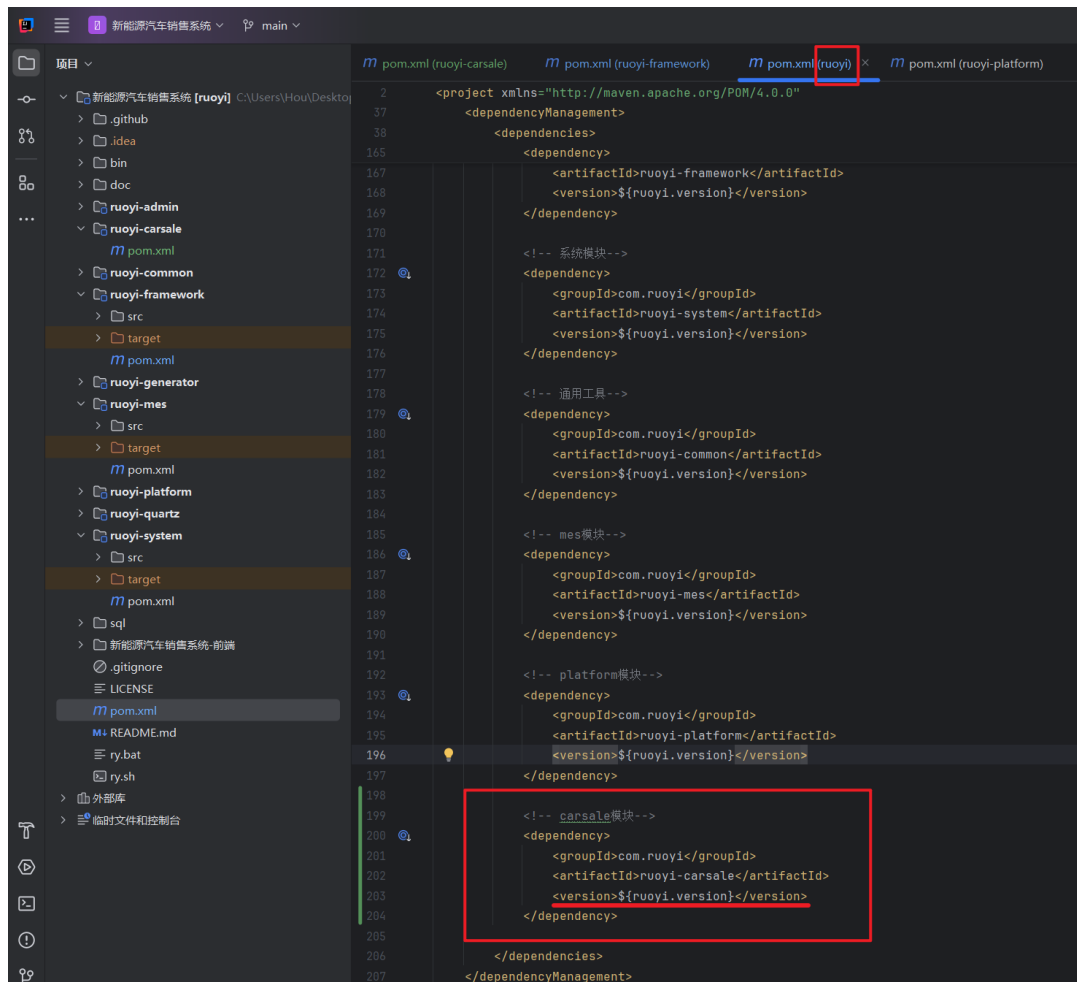
1. 在 业务模块 中加入 common 依赖



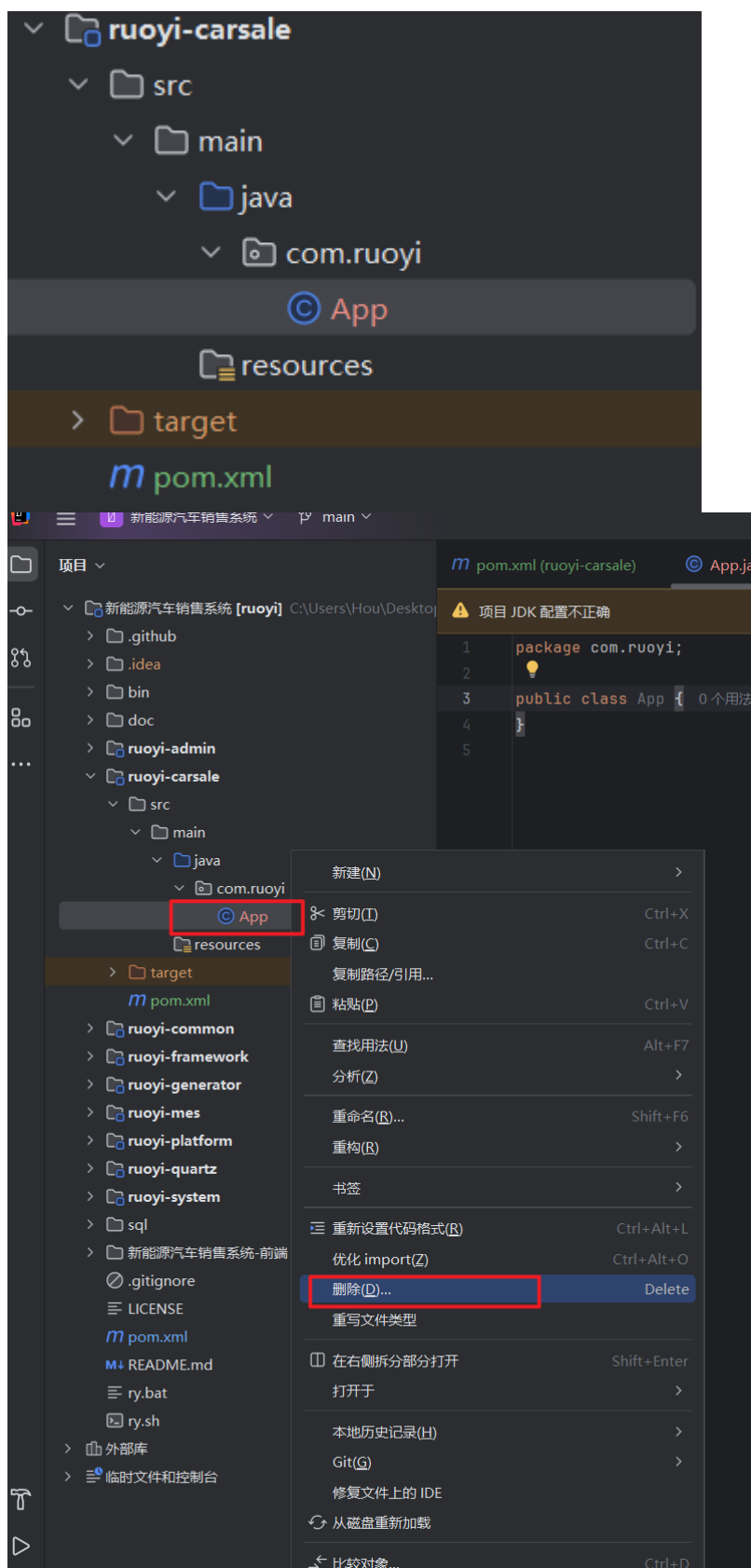
2. 在 framework 模块中依赖 业务模块



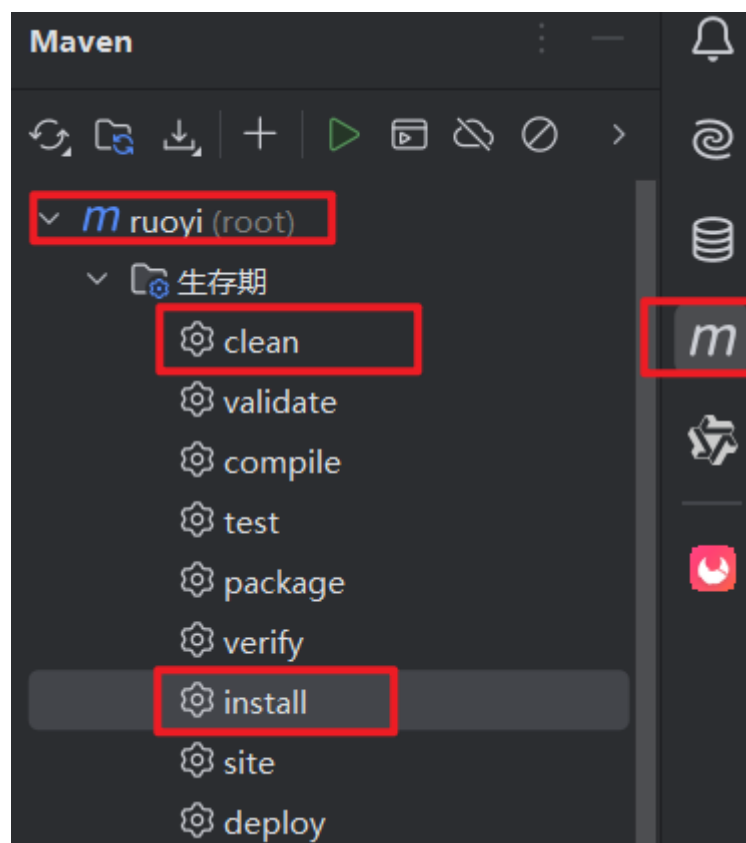
3. 在父工程 pom.xml 中声明依赖版本

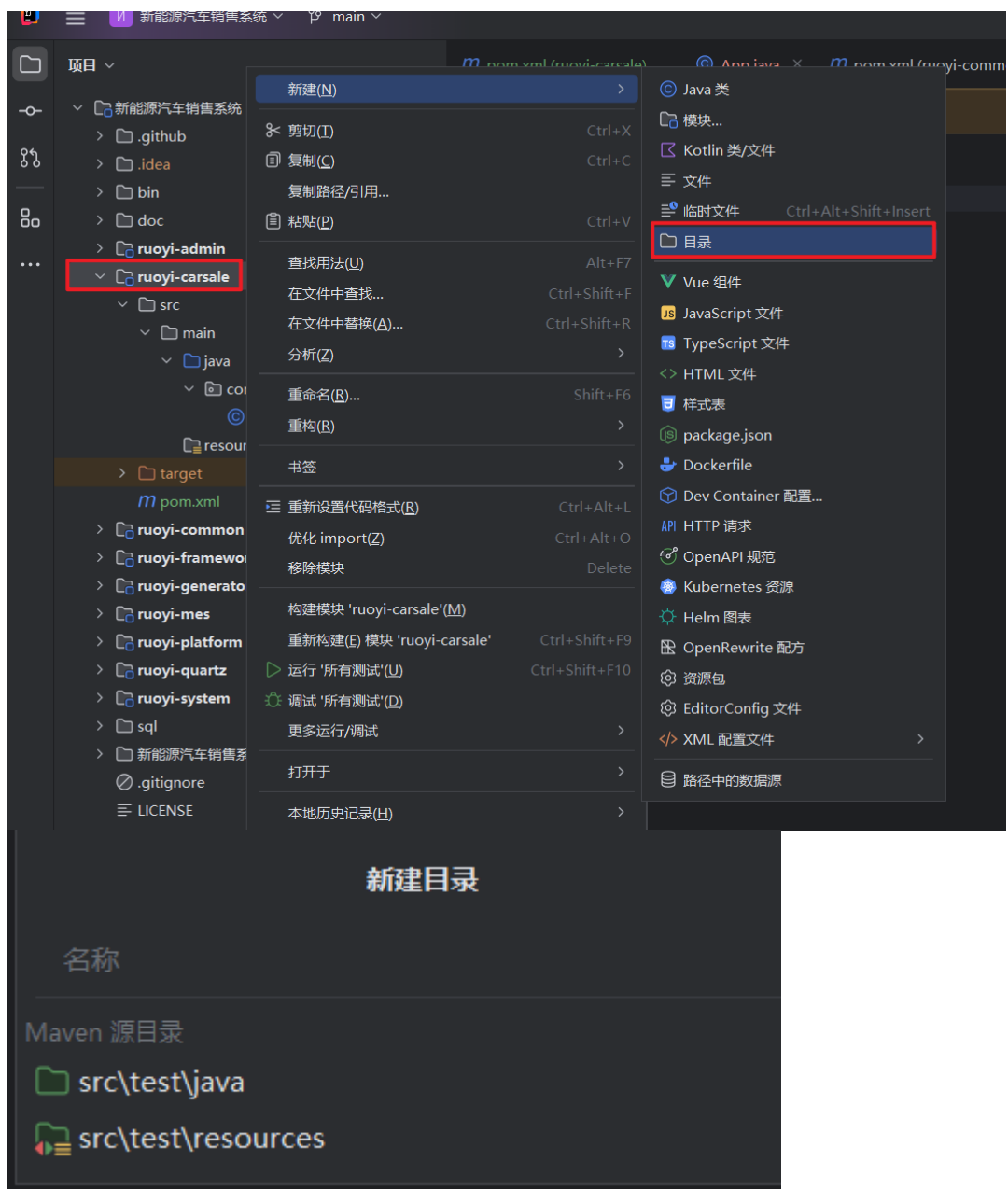


4. 配置文件，按理说你创建好模块之后，只要你的maven识别正确，自动会给你生成这样的路径。注意要把App的java文件给删掉，因为整个项目的入口已经在 `ruoyi-admin` 里面了。



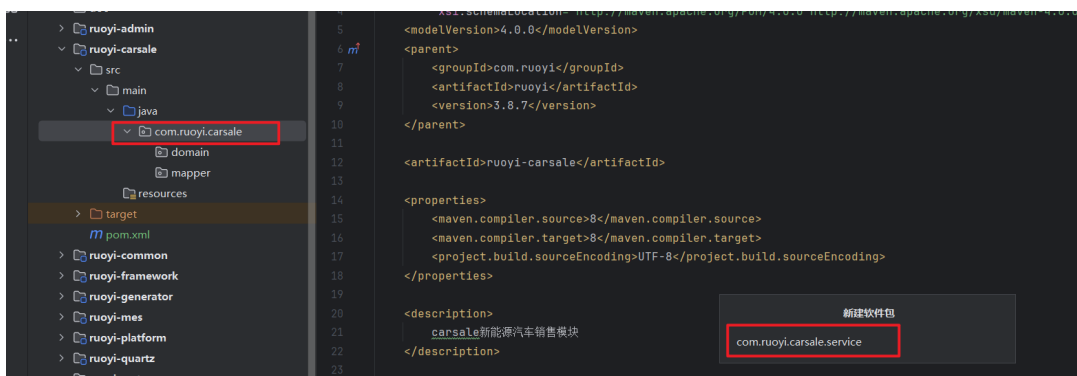
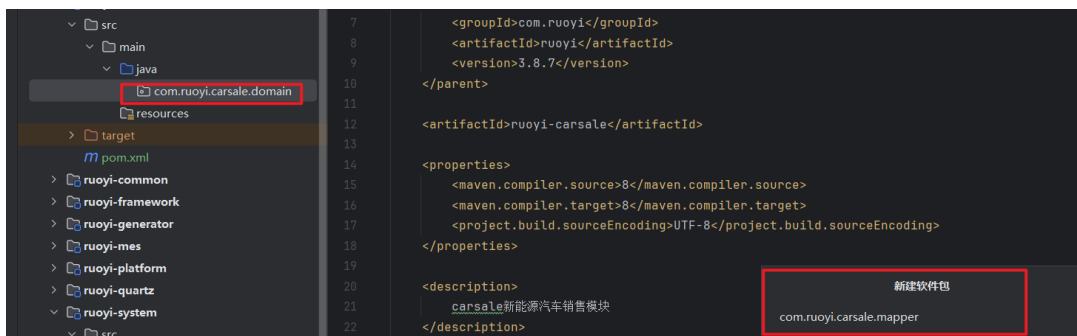
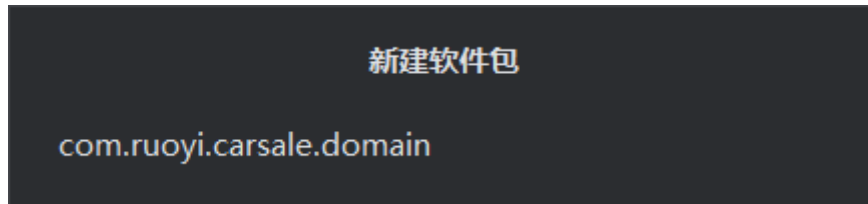
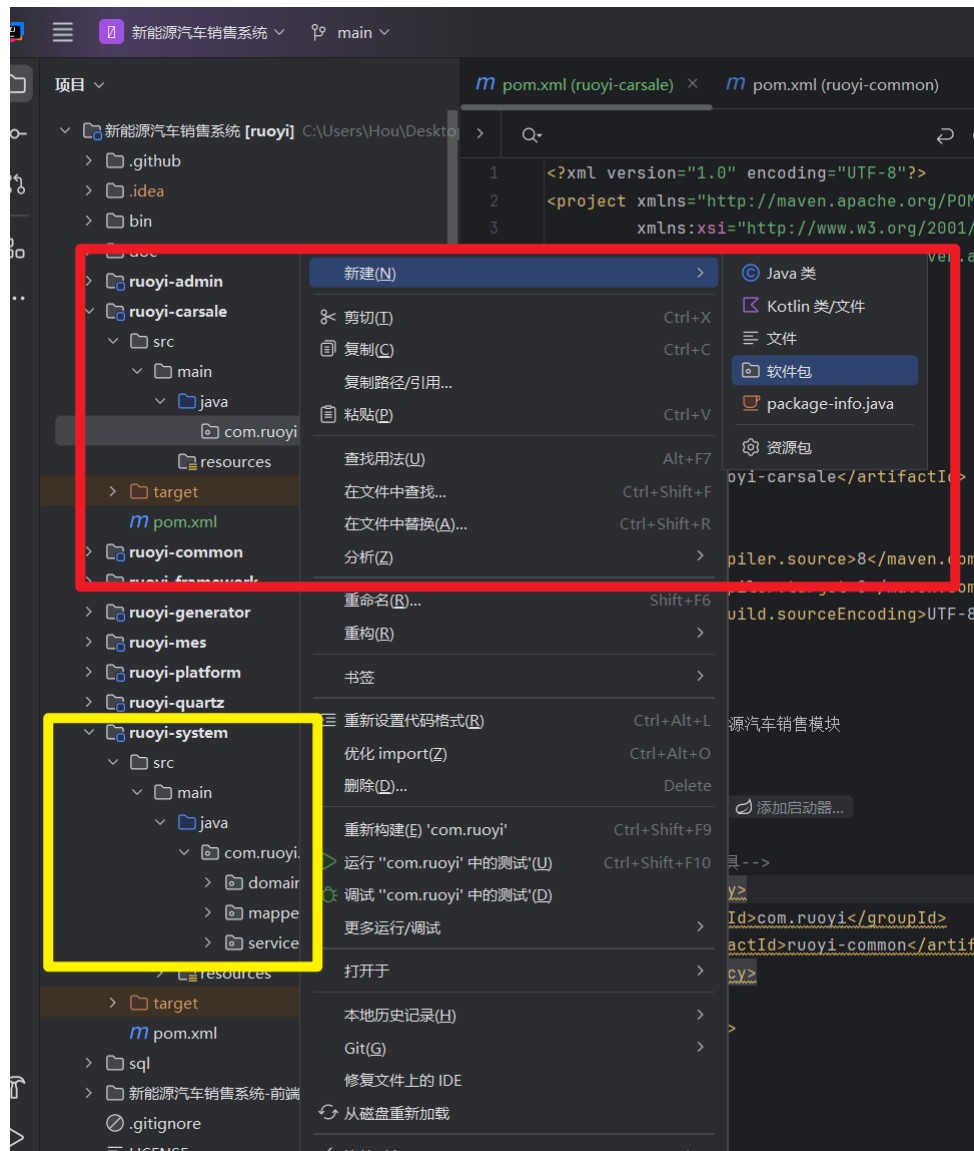
但是如果maven一开始没有识别正确，那你可以手动建立，先清理父工程的旧缓存，在安装整个项目【依次双击 ruoyi 的 clean、install】，此时会将target目录是生成出来，之后在建立文件目录，下面会直接显示maven源目录，直接点就行【图片里面少一个，因为我电脑上已经存在了】



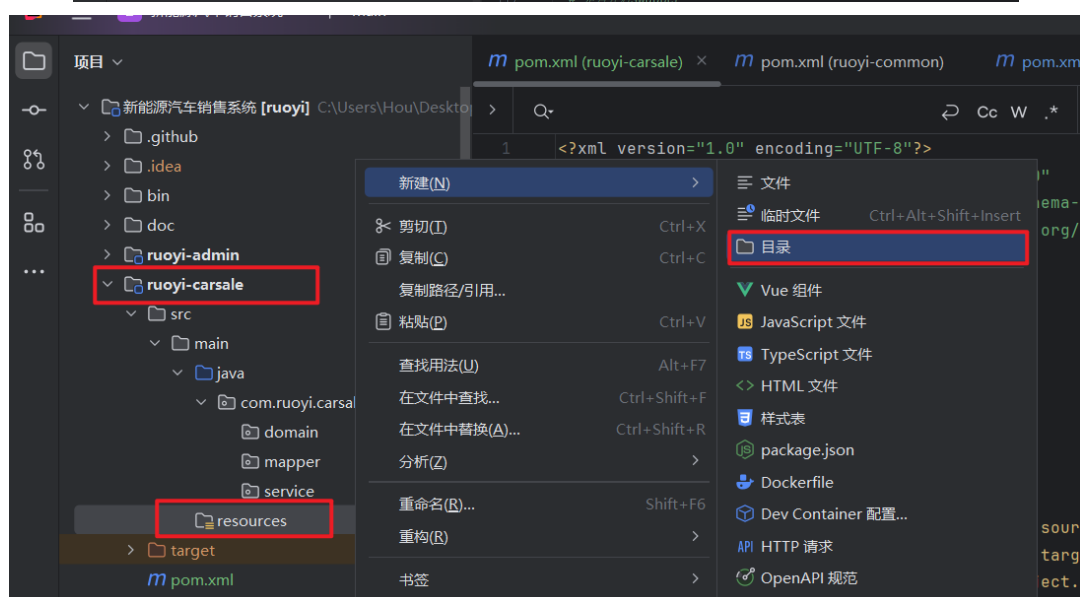
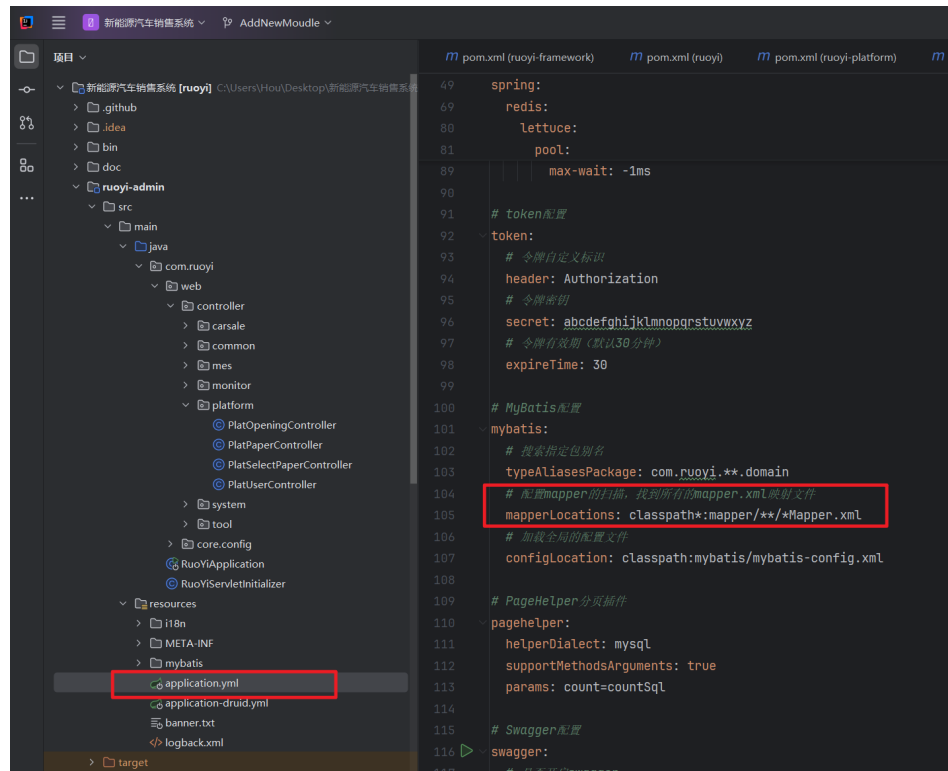


5. 刚刚说过，Controller 在 ruoyi-admin 里面，其他在 业务模块 里面，接下来，按照若依项目的“规则”，建立好业务模块的结构。

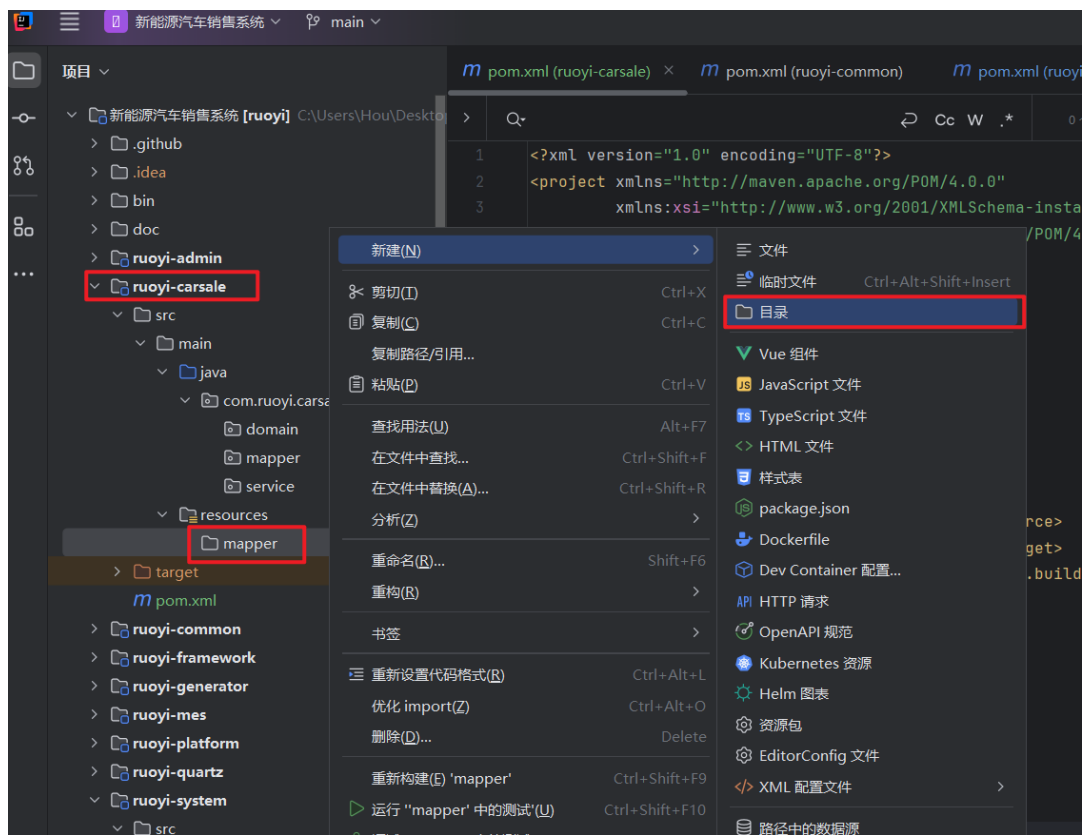
1. 先建立 domain、mapper 和 service



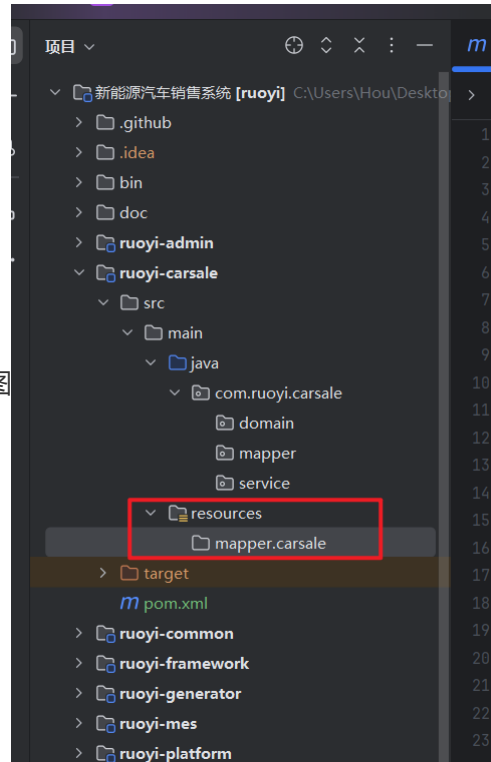
2. 建立Mapper映射文件的结构，注意命名一定要统一，不然得改配置文件。人家若依项目的配置文件里扫描Mapper映射文件的时候，就是：mapper/**/*Mapper.xml，因此你的路径一定要写对。



然后再加一层目录，注意是在mapper下加

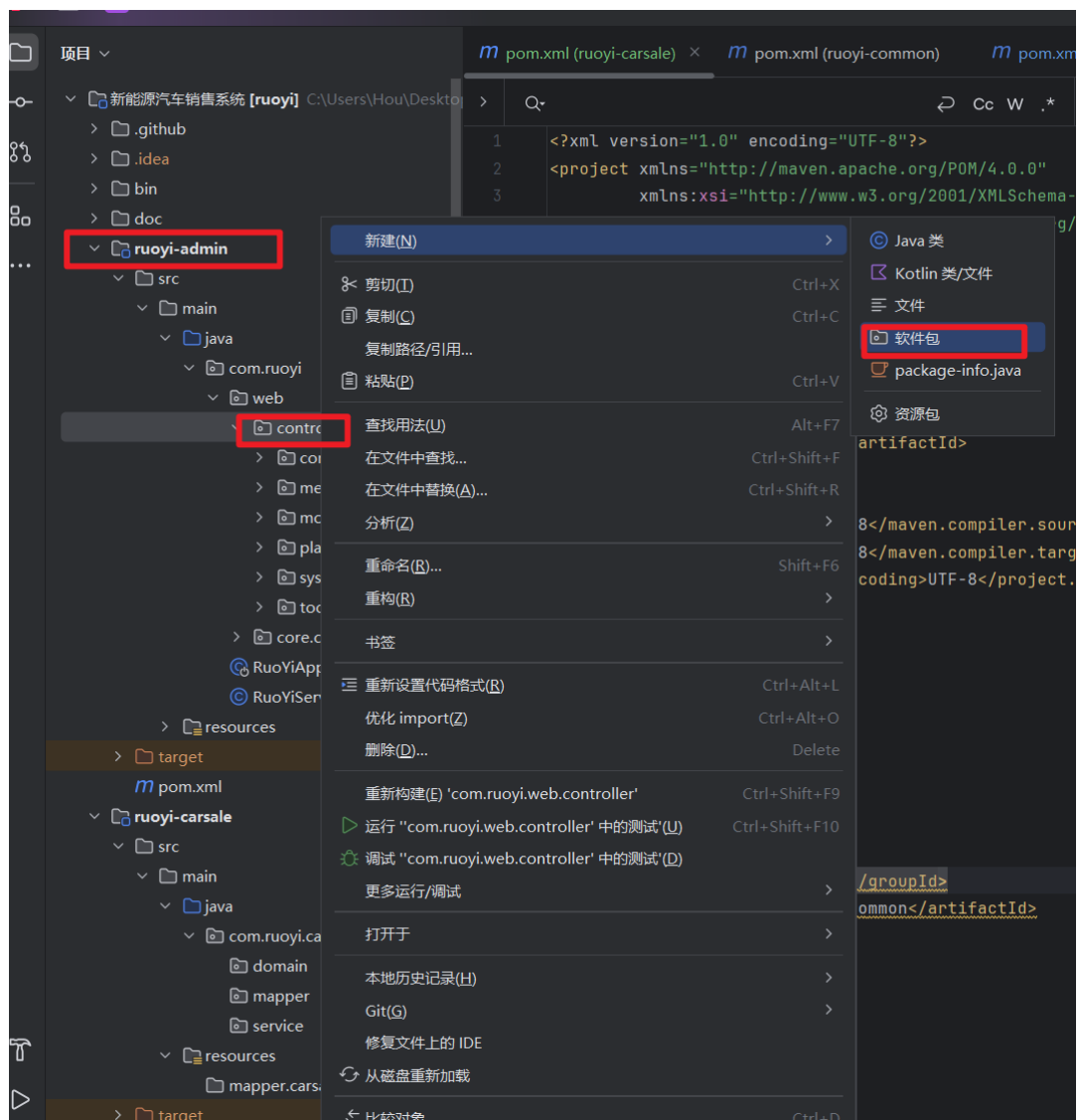


最后建立的结构如图



注意这个mapper.carsale，这个不是目录的名字而是目录的分级，只是这里显示的是点，其实应该是斜杠（mapper / carsale）

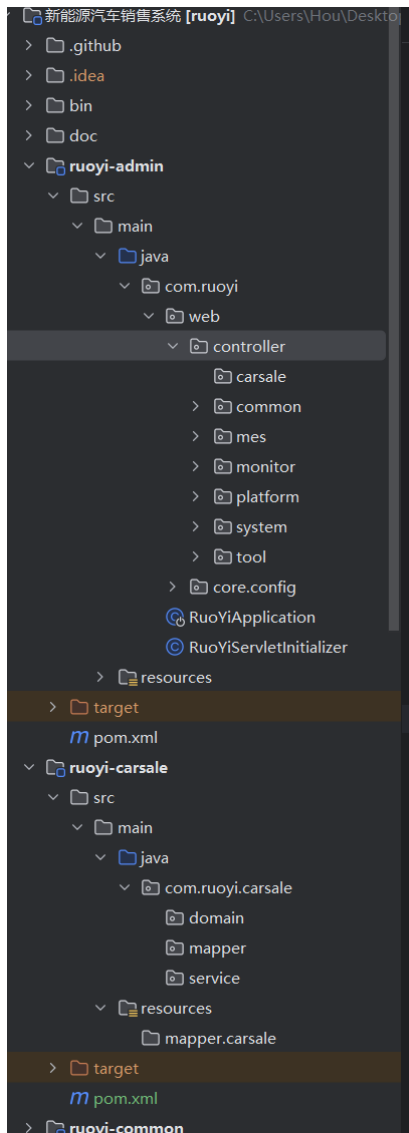
3. 接下来要建立 Controller 层结构，要转到 ruoyi-admin 模块中去



新建软件包

com.ruoyi.web.controller.

至此，模块结构就建立完成了。接下来就是实现MVC各层业务逻辑了

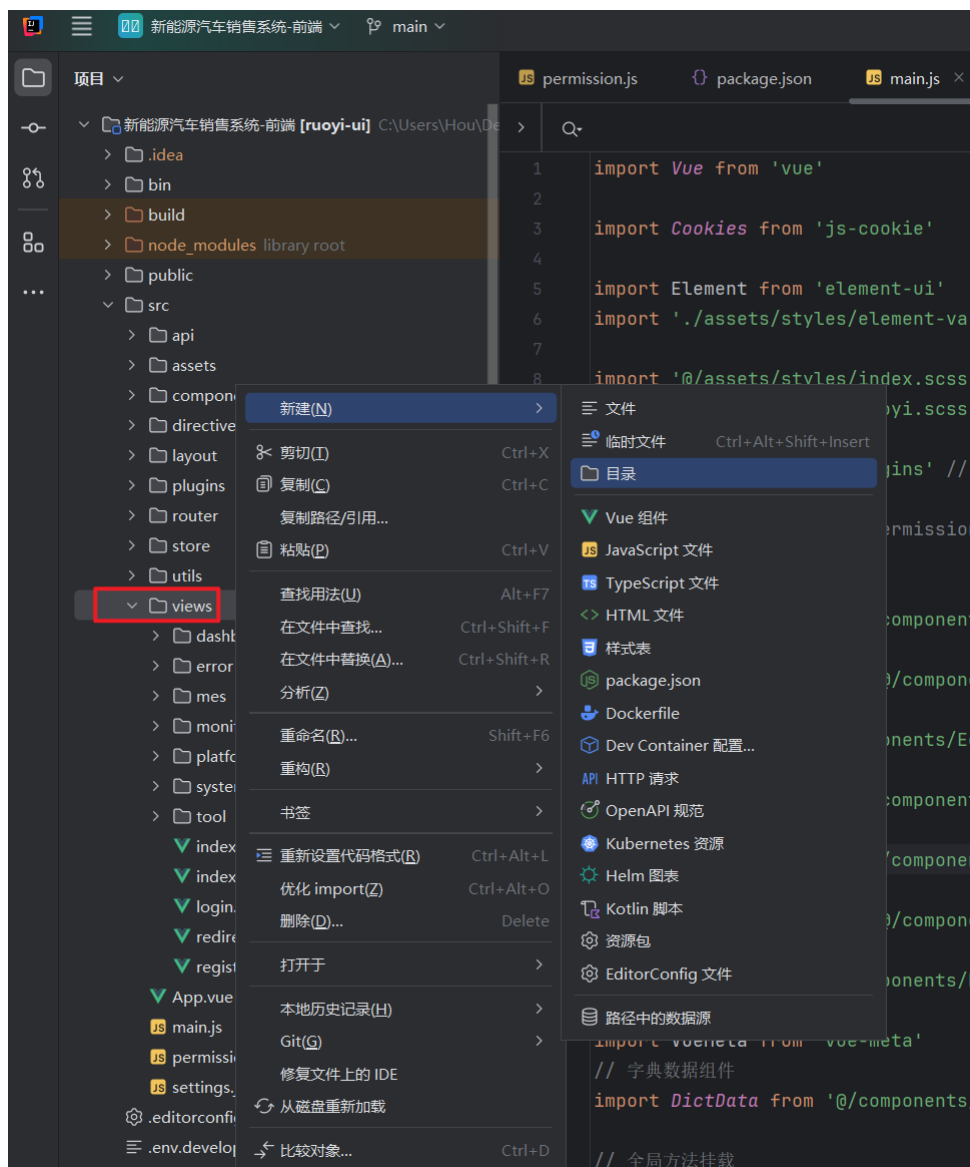


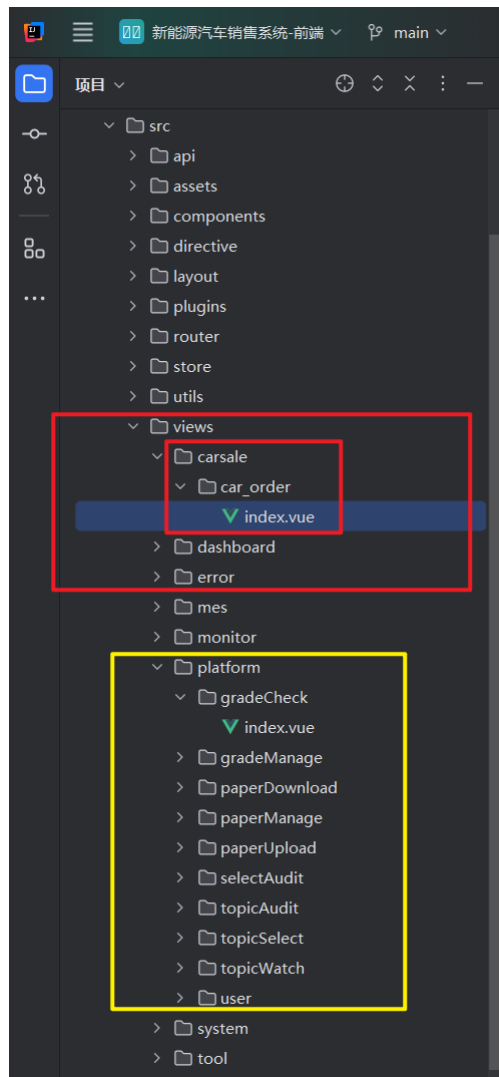
前端结构

关于前端，要做的是：加一个view页面的 index.vue 文件、一个调用后端api的js文件 即可

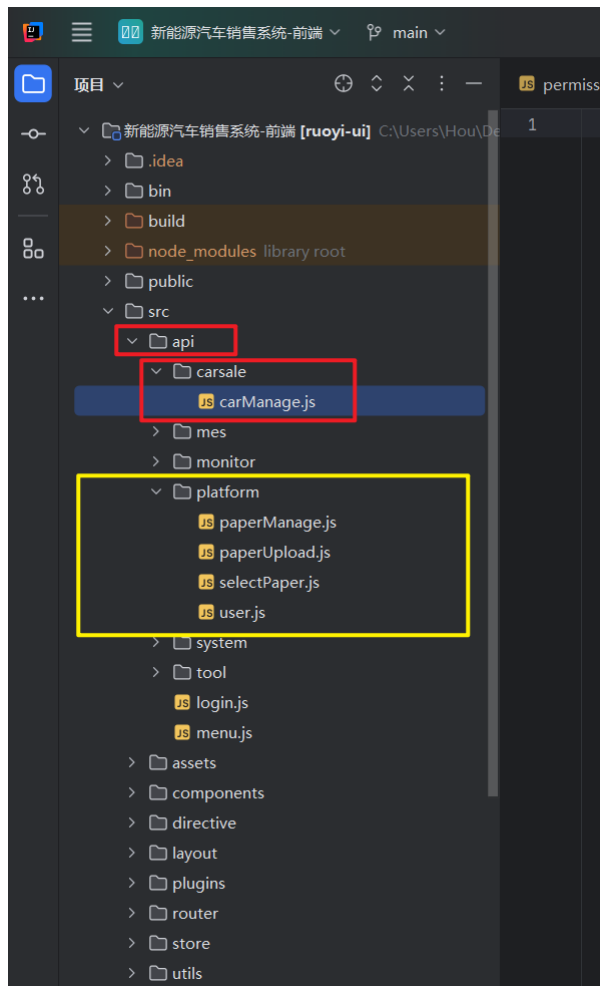
若依项目还是比较方便的，路由可以直接在ui界面添加菜单，之后路由直接在数据库中动态加载过来就可以

1. 在 views 目录下新增业务逻辑所需要的文件夹 carsale，然后在 carsale 文件夹下面增加各种功能所需要的目录。比如说，carsale 业务逻辑中有一个“用户购车”的功能，因此在 carsale 目录下建立一个 car_order 的目录，然后把“用户购车”的前端页面代码 index.vue 放在这个文件夹下。其他功能同理【注意，建议所有的功能下页面都命名为 index.vue，可以参考 platform】





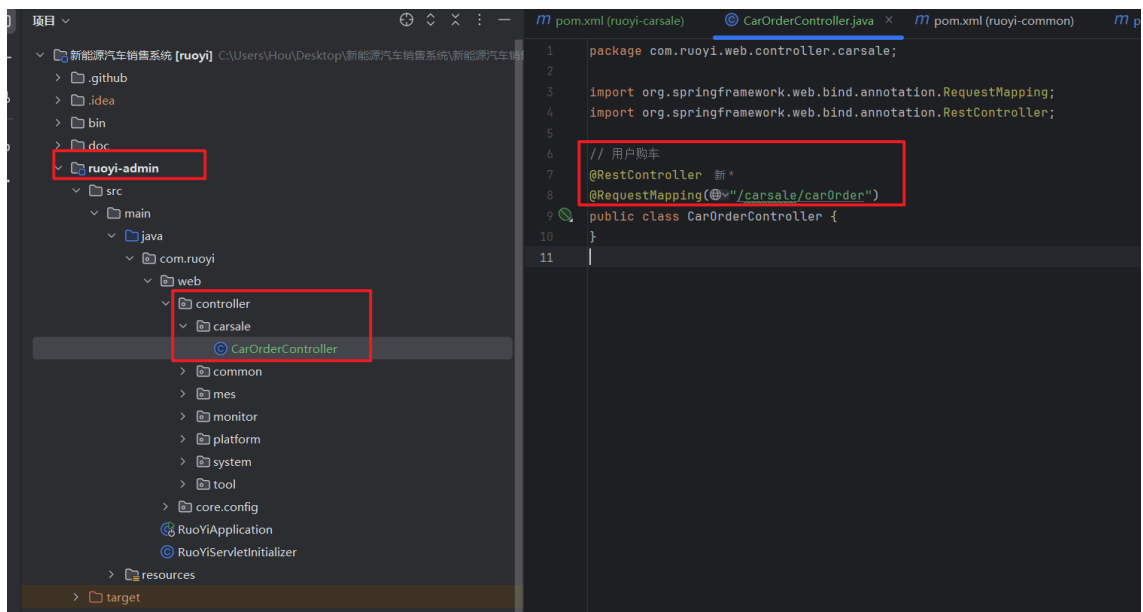
2. 前端代码写好后，调用后端接口。转到 `api` 目录下，建立需要的文件【可以参考 `platform`】



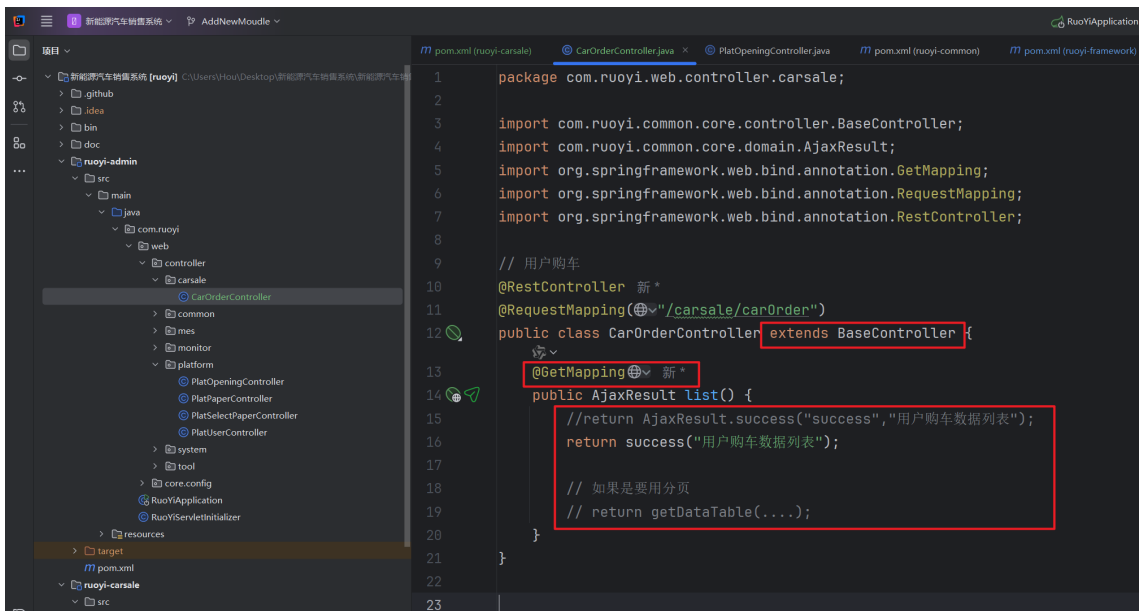
至此，前端结构也都弄完了。

功能实现_1

1. 首先在后端建立一个“用户购车”功能的Controller

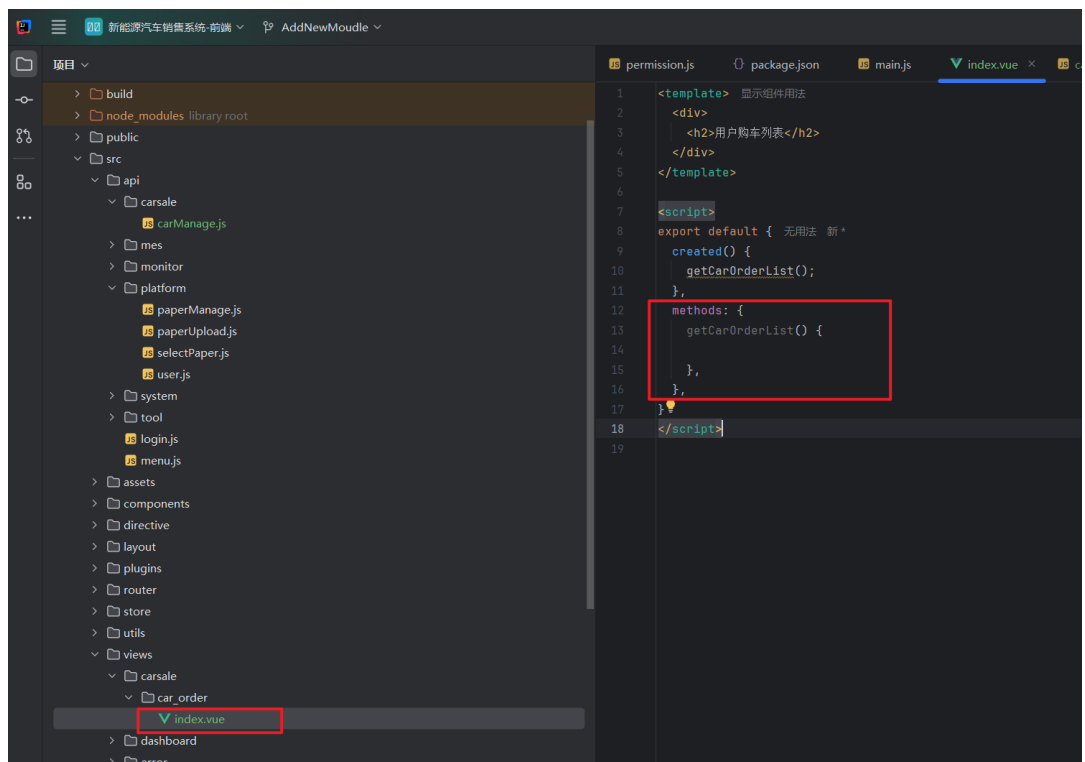


注意，若依封装好的统一的相应接口的对象叫 `AjaxResult`，因此要返回该类型。这里可以通过继承 `BaseController`，直接使用 `success()` 方法

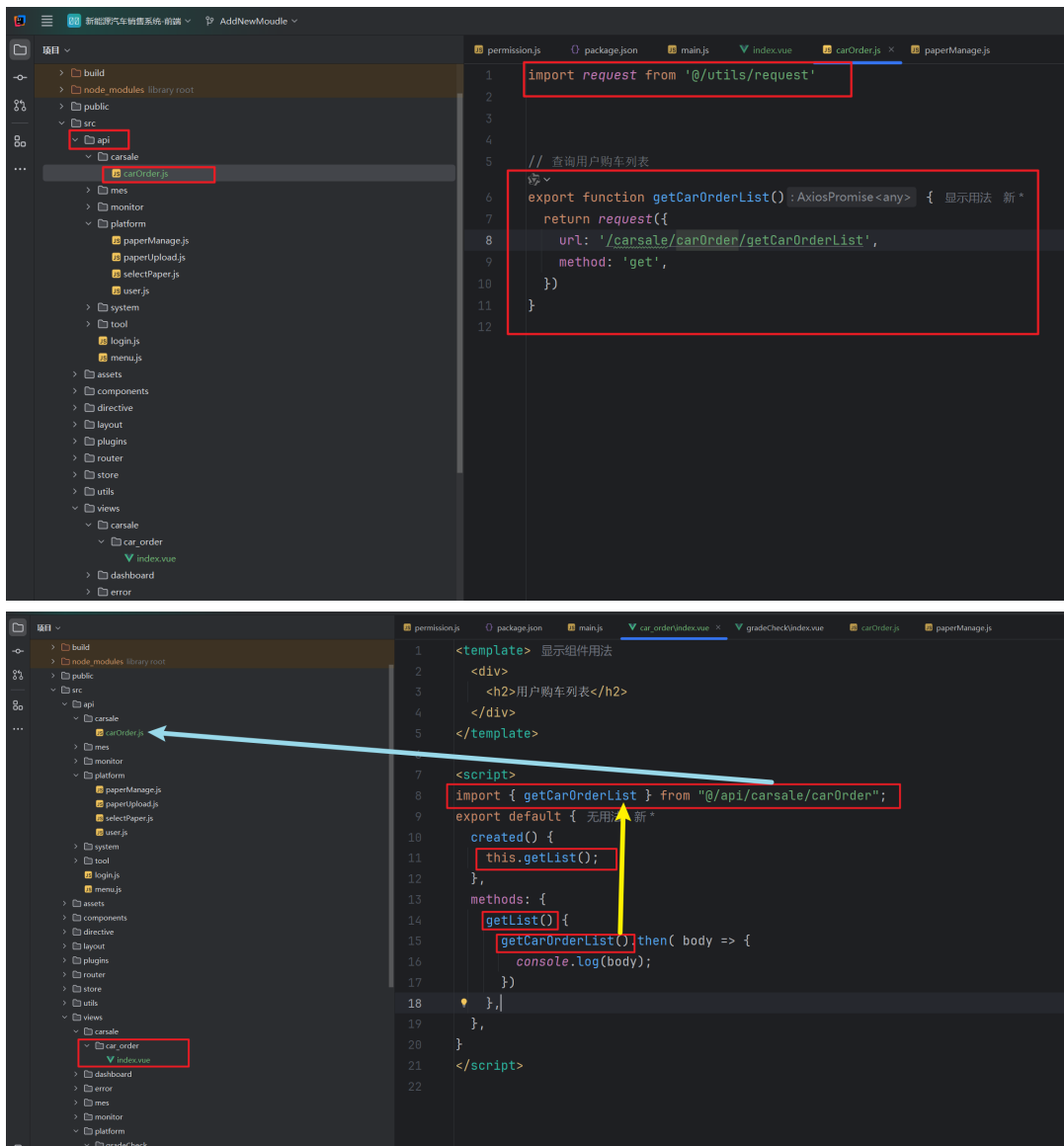


2. 为了测试，这里先写一个简易的前端，顺便写一下前端应该如何规范

1. 首先写一个简易的index.vue文件

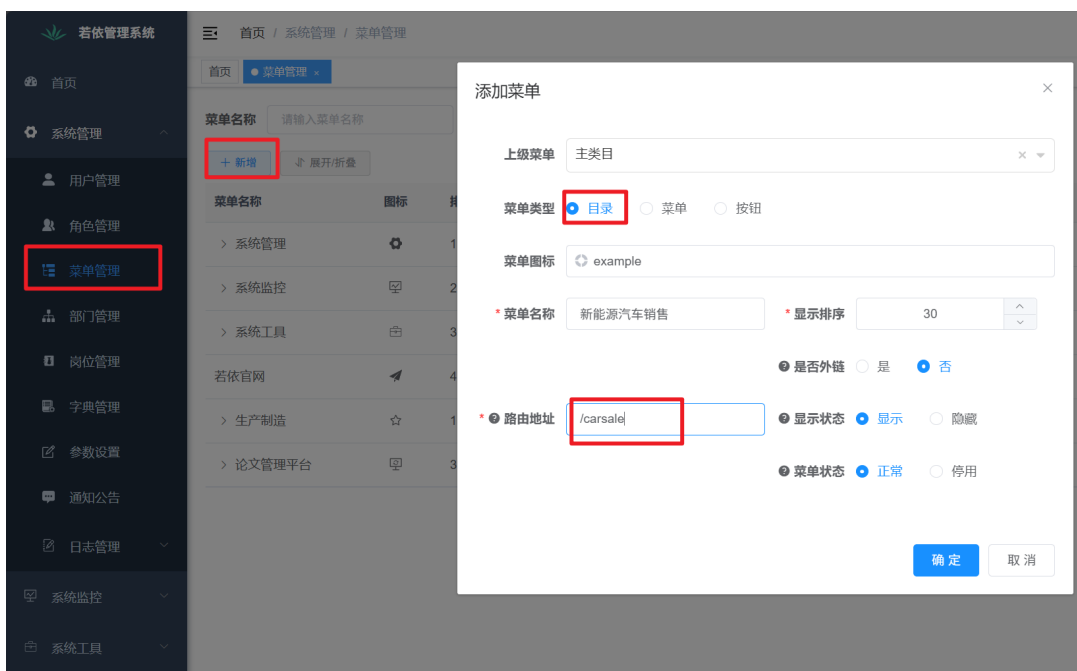


2. 注意关于后端调用的代码全部在api目录下封装着，同时若依项目把所有发送请求的函数都封装在request里了，直接引入，然后完成具体功能。同时，别忘了在index.vue里面去引入api中的文件



3. 设置路由，直接在UI界面设置，更新数据库，然后路由会根据数据库自动更新。

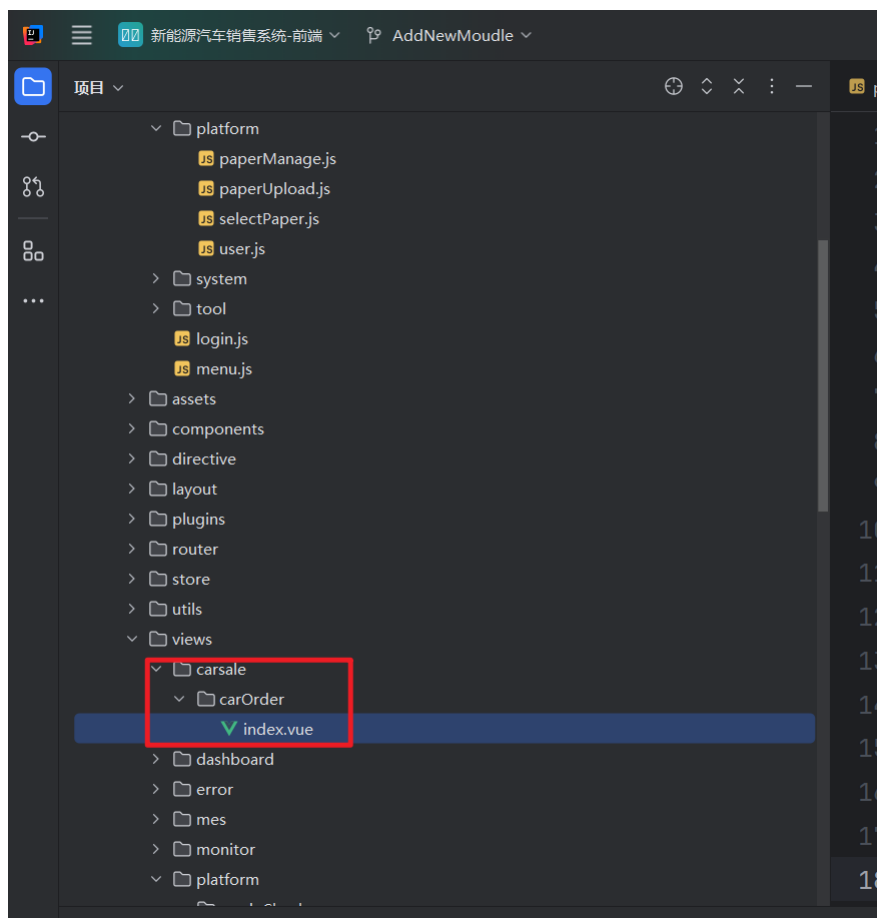
1. 首先，增加一个目录：新能源汽车销售，路由设置为 "/carsale"



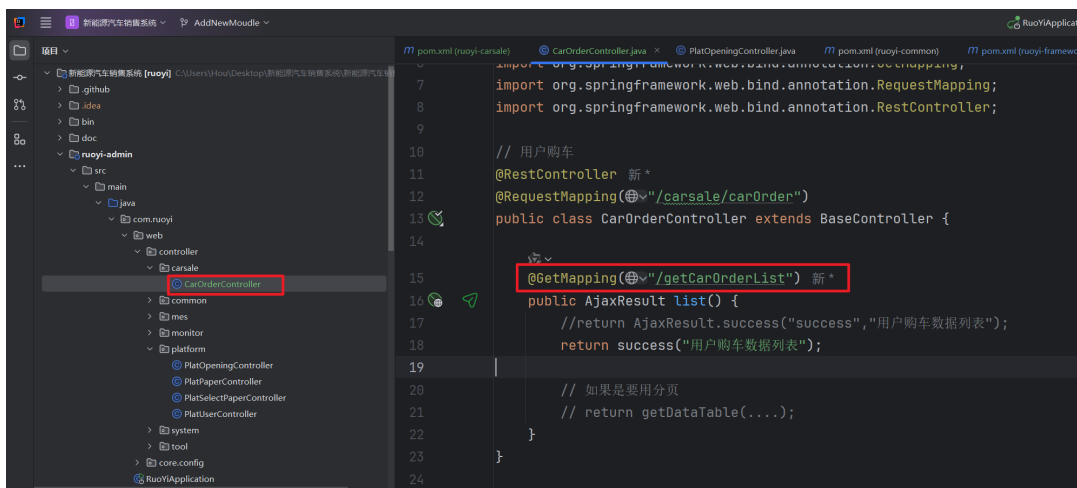
2. 在新增菜单基础上，再增加各种功能的菜单项，菜单项的路由地址和组件路径要写对



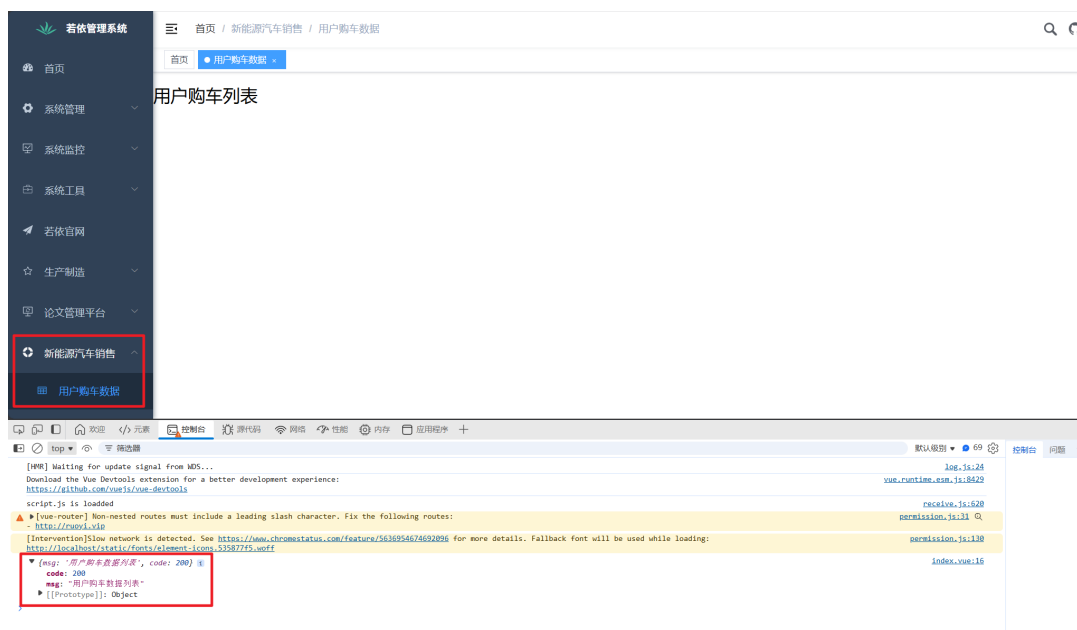
组件路径对应的就是前端相应页面 `index.vue` 的路径（从views目录开始）。为了方便，我把这个car_Order目录名字改为了carOrder



这里补一个后端的東西，路径没给全，导致找不到对应的函数



刷新前端页面，同时打开控制台，可以看到后端返回来的东西



功能实现_2

继续完善"用户购车"功能，这里没有数据，我就拿mesProject的数据库来举例，完成carOrder的操作。首先要知道各层负责干什么，一个一个建立，后续熟悉之后可以用若依的代码生成功能

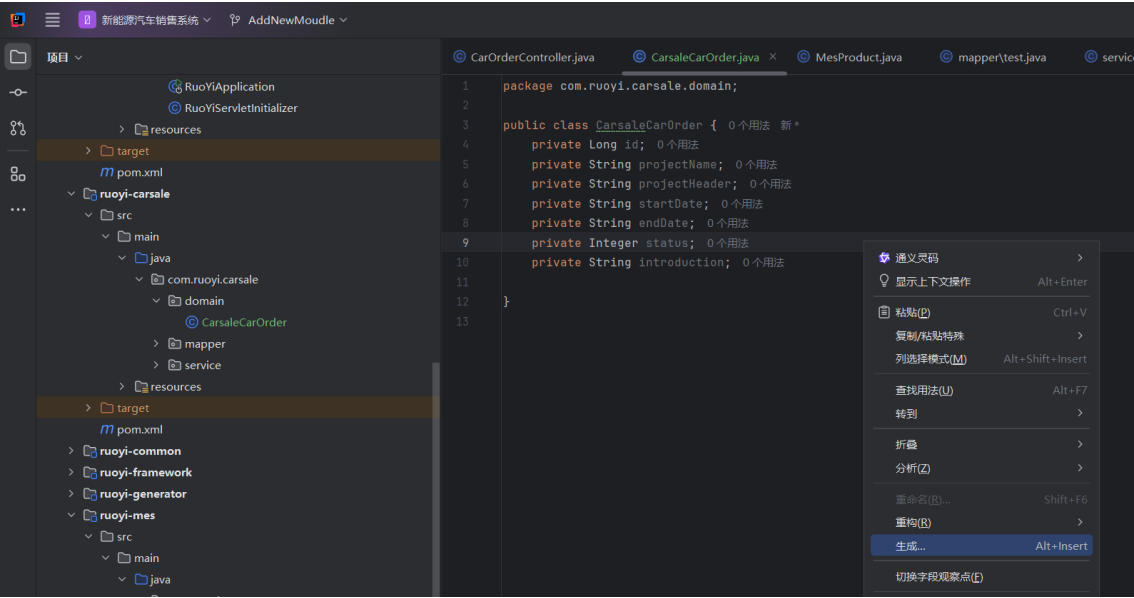
层次名称	对应餐厅角色	技术职责
Controller (控制层)	服务员	负责接收客人的请求（API接口），检查参数对不对，最后把结果传回给客人。
Service (业务层)	主厨	核心逻辑所在。负责处理复杂的业务（比如算账、库存判断、发通知），它不关心数据怎么存，只关心“怎么做”。
Domain (实体层)	菜谱/订单详情	定义了数据的长相。比如一个“用户”包含 ID、用户名、密码，这些属性都定义在这里。
Mapper (持久层)	仓库管理员/采购员	专门负责和数据库打交道。执行 SQL 语句，把数据库里的数据取出来，或者把新数据存进去。

1. domain层

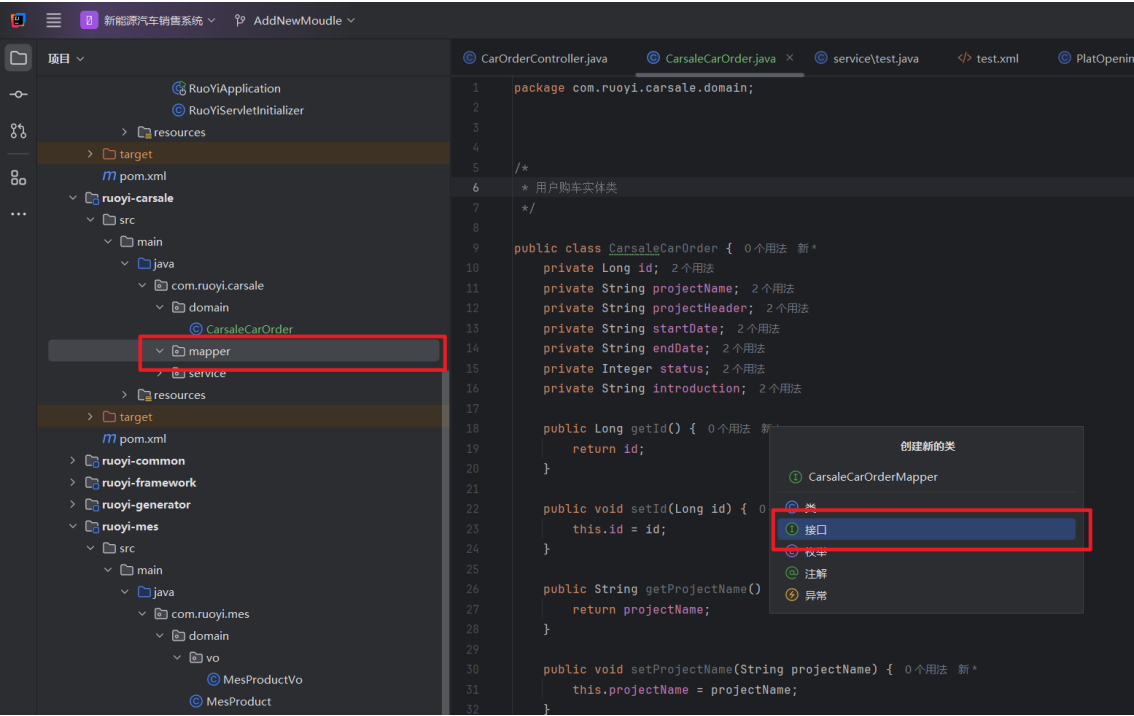
首先先明确数据库怎么设计的，这里因为以mesProject举例，所以对应的数据库长这样：

id	project name	project header	start date	end date	status	introduce
5	2022-05生产项目	斯蒂芬	2022-05-24 00:00:00	2023-05-06 00:00:00	1	(Null)
6	2022-06生产项目	勒布朗	2022-06-14 00:00:00	2023-05-31 00:00:00	1	(Null)
9	2022-07生产项目	凯文	2022-07-01 00:00:00	2023-07-01 00:00:00	1	(Null)
13	2022-08生产项目	凯里	2022-08-01 00:00:00	2023-08-01 00:00:00	1	(Null)
14	2022-09生产项目	德里克	2022-09-01 00:00:00	2023-09-01 00:00:00	1	(Null)
15	2022-10生产项目	德克	2022-10-01 00:00:00	2023-10-01 00:00:00	1	(Null)
16	2023-05生产项目	贾森	2023-05-01 00:00:00	2024-05-01 00:00:00	1	(Null)
17	2023-06生产项目	克里斯	2023-06-01 00:00:00	2023-08-01 00:00:00	1	(Null)
18	2023-07生产项目	扬尼斯	2023-07-13 00:00:00	2024-08-01 00:00:00	1	(Null)
19	2023-08生产项目	乔尔	2023-08-01 00:00:00	2024-08-01 00:00:00	1	(Null)
20	2023-09生产项目	尼古拉	2023-09-01 00:00:00	2024-09-01 00:00:00	1	(Null)
21	2023-10生产项目	贾马尔	2023-10-01 00:00:00	2024-08-31 00:00:00	1	(Null)
22	2022-11生产项目	克里斯	2022-11-01 00:00:00	2022-11-30 00:00:00	1	(Null)
23	2022-12生产项目	安吉	2022-12-01 00:00:00	2022-12-31 00:00:00	1	(Null)
24	2023-01生产项目	瑞安	2023-01-01 00:00:00	2023-01-31 00:00:00	1	(Null)
25	2023-02生产项目	安吉洛	2023-02-01 00:00:00	2023-02-28 00:00:00	1	(Null)
26	2023-03生产项目	德里克	2023-03-01 00:00:00	2023-03-31 00:00:00	1	(Null)
27	2023-04生产项目	拉塞尔	2023-04-01 00:00:00	2023-04-30 00:00:00	1	(Null)

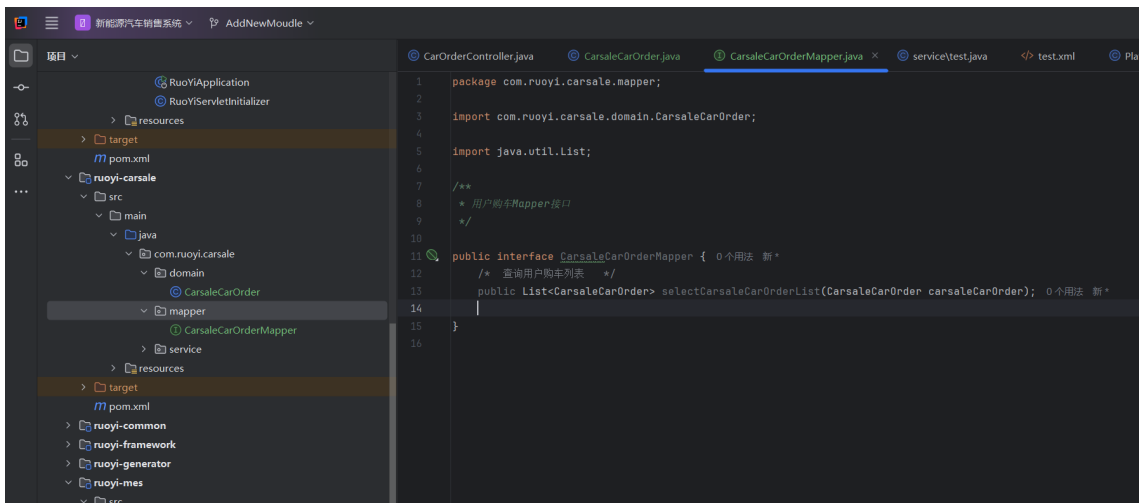
因此实体类里面定义的数据结构要包含这几个属性，写好这几个属性之后，右键 ---> 生成 ---> 直接给所有属性生成Get、Set方法



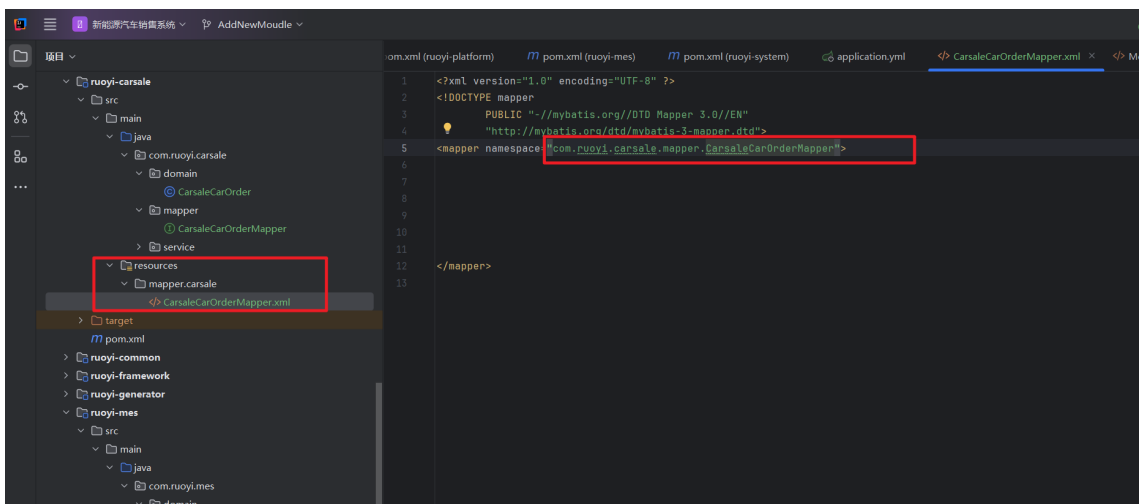
2. mapper接口【注意与mapper映射文件作区分，mapper映射文件是放在resource目录里的xml文件】，注意是创建Java接口，而不是Java类



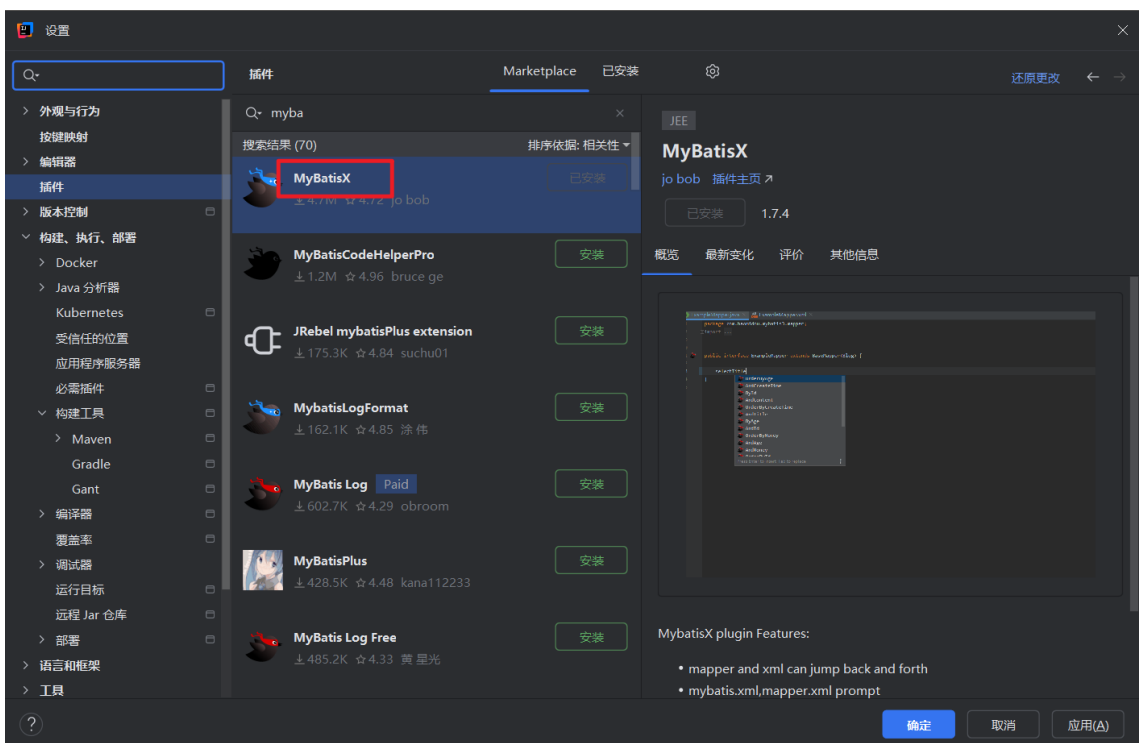
Mapper接口中主要是定义一些方法



3. 建立Mapper映射文件，然后随便找一个其他业务模块的映射文件粘贴过来，把里面东西删掉，只保留上面的信息。注意把映射路径改对。将SQL语句全部写在Mapper映射文件中。

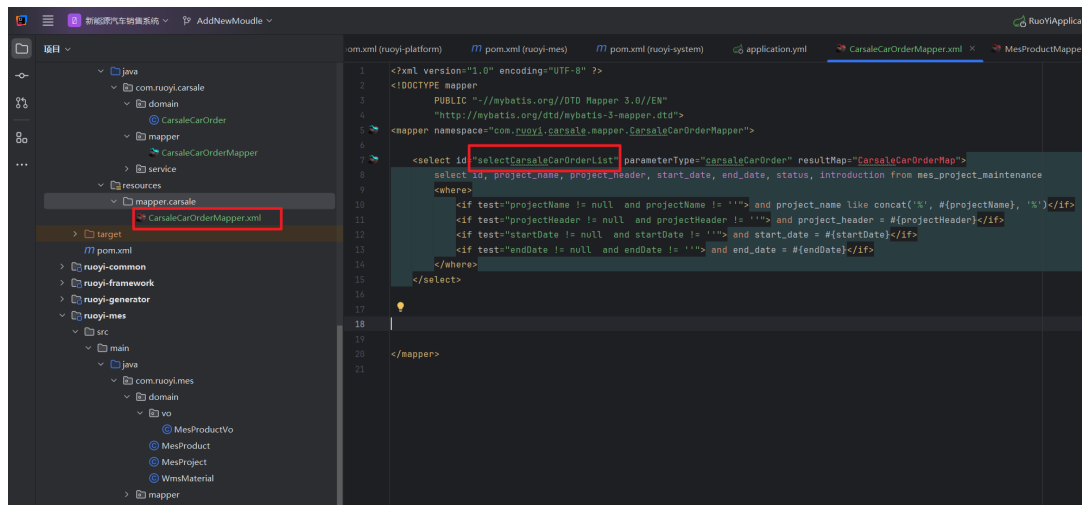
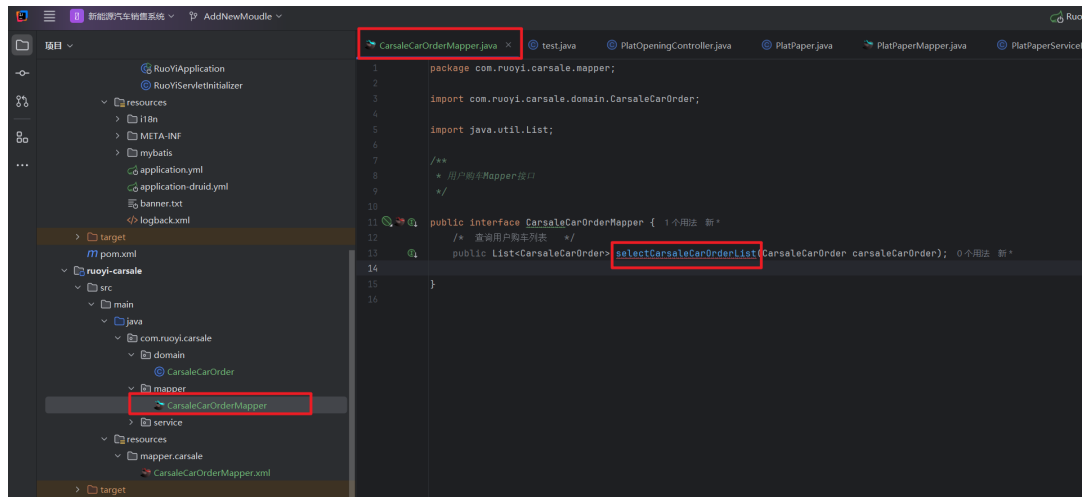


为了使Mapper映射文件和Mapper接口对应起来比较方便，可以安装一个插件：MyBatisX

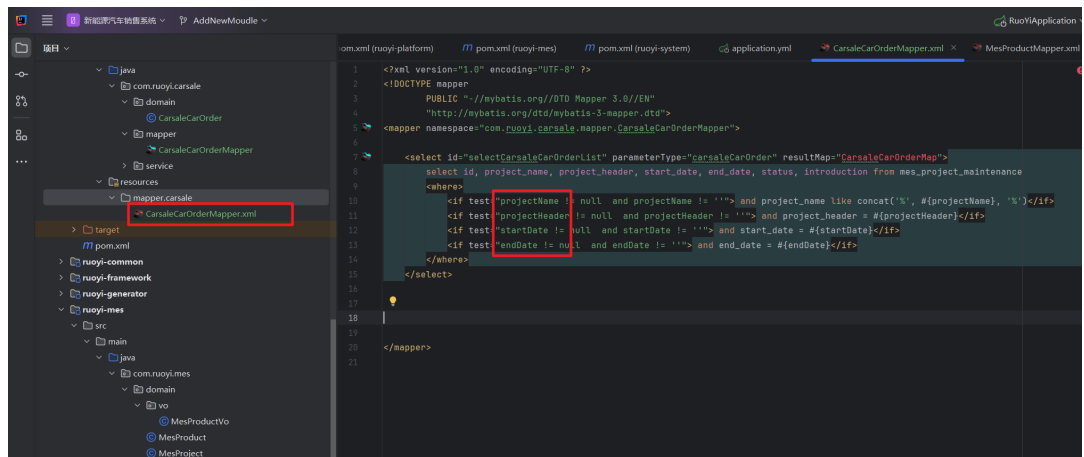


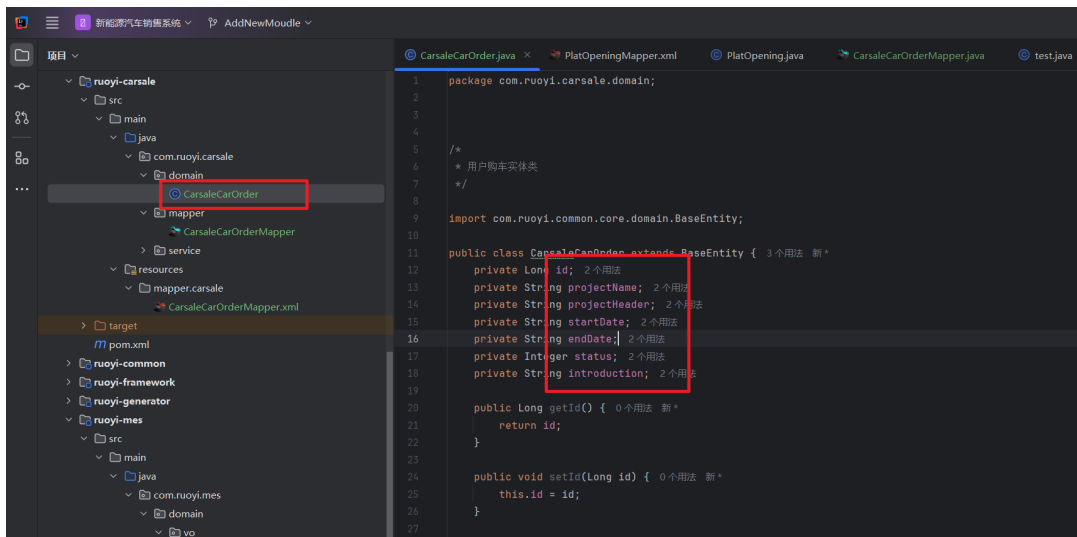
这里有几个映射名词要对应起来

1. Mapper接口中的函数名字要跟Mapper映射文件中，id名字保持一致，不然找不到映射了【这里都命名为了 `selectCarsaleCarOrderList`】

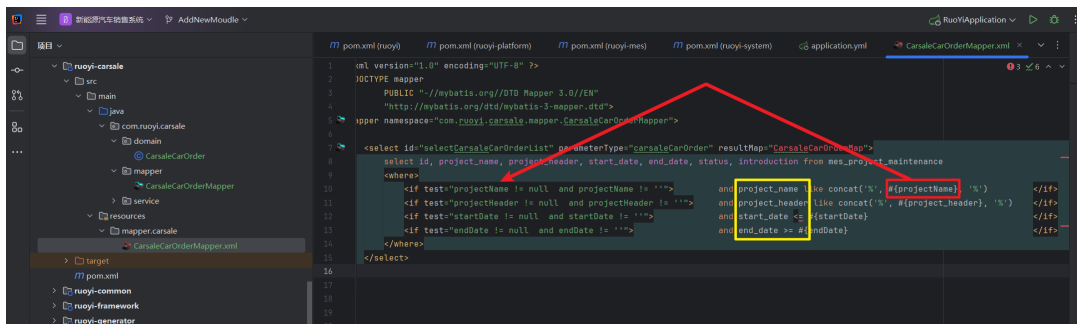


2. SQL语句中的test参数，对应的是实体类里面定义的属性名字，而不是数据库中的字段名

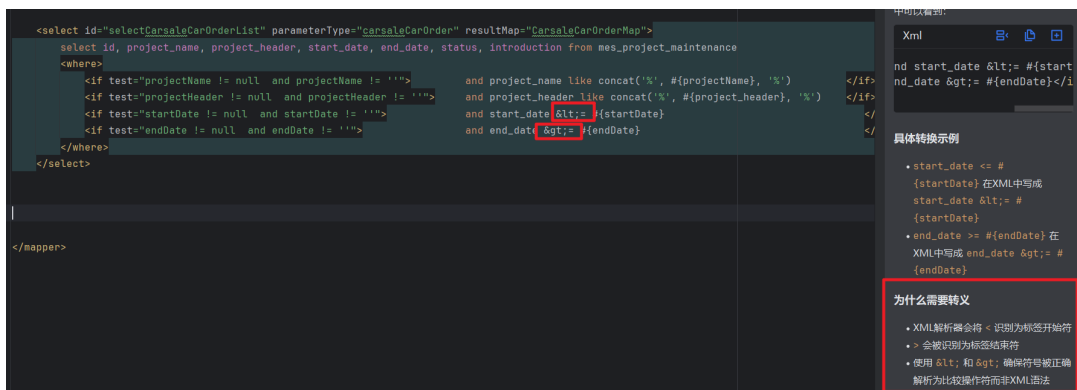
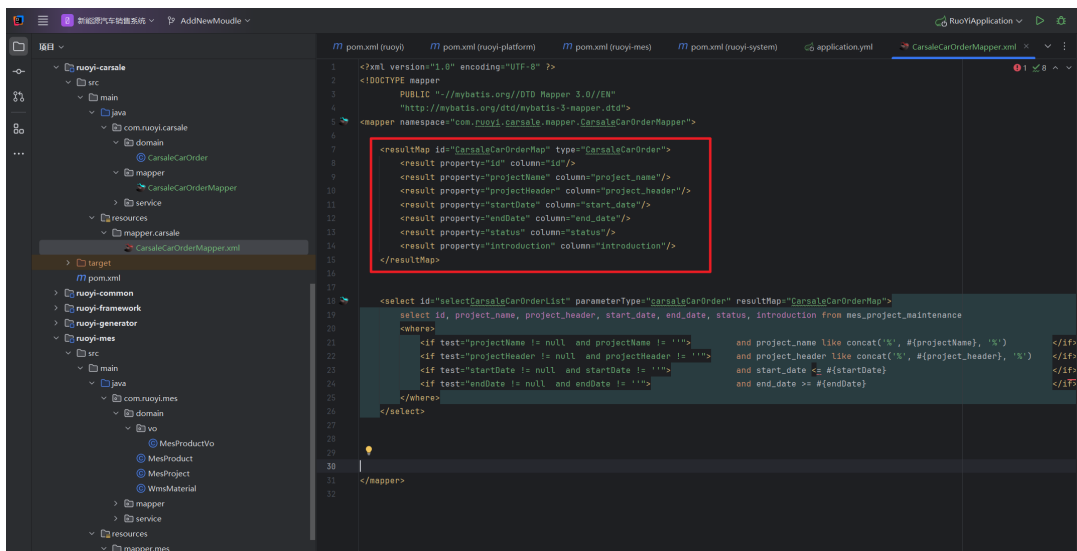




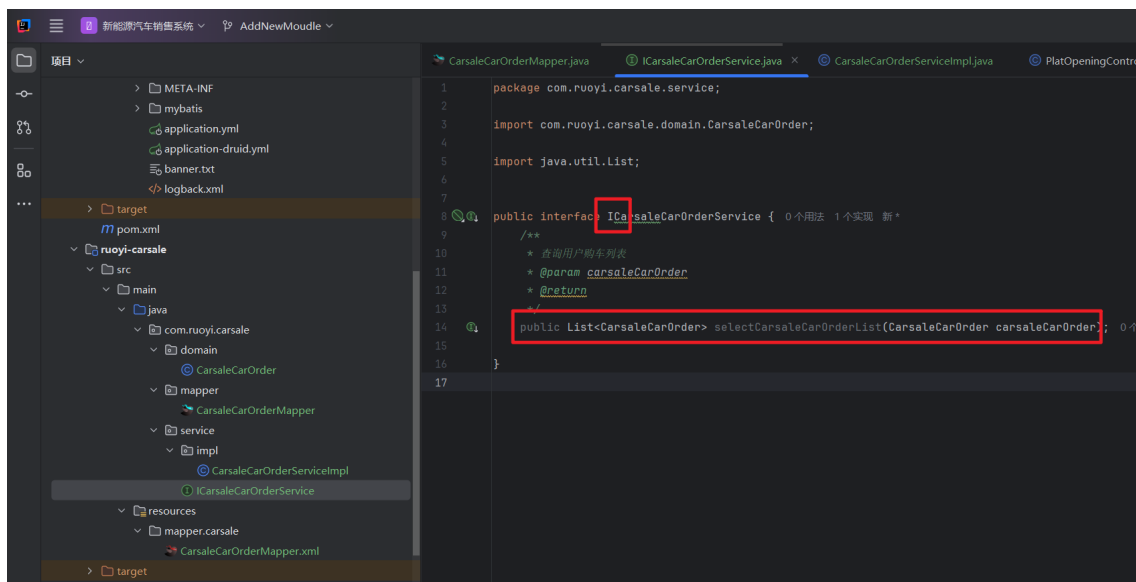
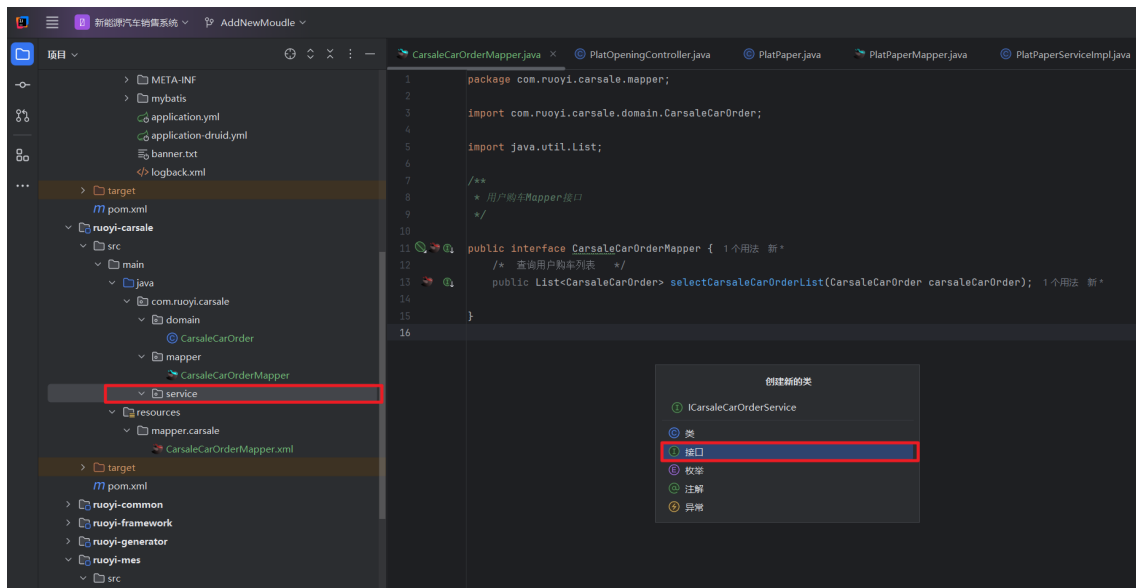
3. SQL语句中的具体找哪个，才是数据库中的字段名【黄色方框是数据库中的字段名，红色箭头是实体类里面的属性名】



4. 此外，需要将数据库查询结果映射到"CarsaleCarOrder"类型的Java对象上，因此要写一个结果映射配置。这些东西，你只要将 建表的sql语句 以注释的方式写在上面，通义灵码会直接给你补全

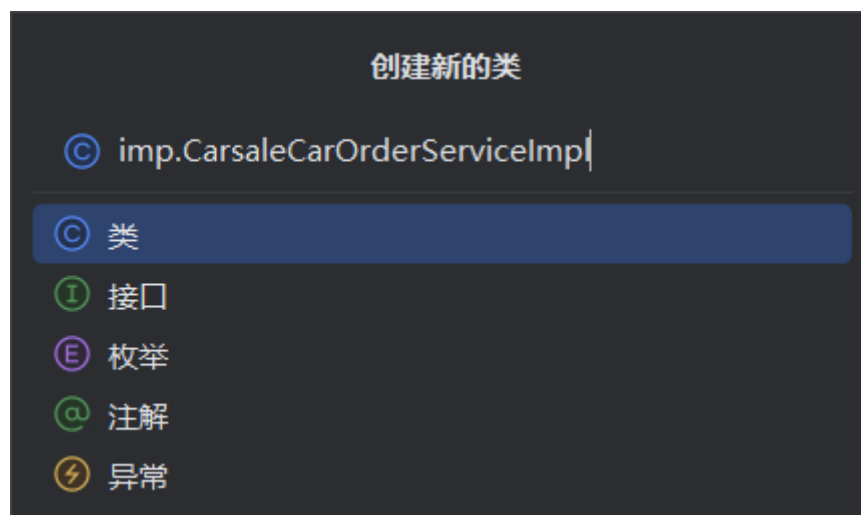


4. 建立Service接口，若依这个项目给Service接口命名规范前面都加了个大写的"I"，我也不知道为啥，大写"I"可能是接口的意思，按着人家的规范来吧，里面定义需要实现的函数

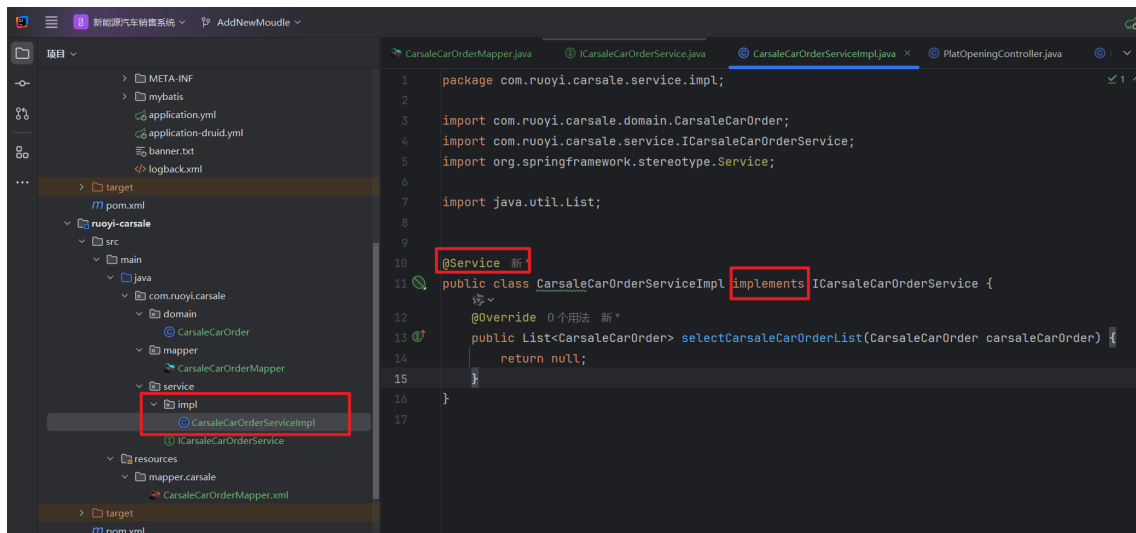


5. 建立Service接口的实现类，可以理解成：Mapper接口的实现类在Mapper映射文件里完成了、Service接口的实现类要建立对应的Java文件

根据若依的套路，建立对应接口的实现类【右击service目录 ---> 新建Java ---> 这里要多建一层目录，直接命名成impl.xxxx就行】



这里要注意两个东西：①这是个Bean组件，要写@Service注解，②要加Implement关键字，重写Service接口中的方法



第二个图：关于 `impl` 文件夹

你提到的 MVC 架构（Controller -> Service -> Mapper -> Domain）是完全正确的。但在实际的大型项目（如若依）中，**Service 层**通常会被拆成两部分，放在不同的包里：

1. `impl` 是哪一层的？

它依然属于 **Service 层**（业务逻辑层）。

2. 为什么要多出一个 `impl` ？

在若依的规范里：

- **service 目录**：存放的是 **接口 (Interface)**。文件名通常以 **I** 开头（如 `ICarsaleCarOrderService.java`）。
- **service/impl 目录**：存放的是 **具体的实现类 (Implementation)**。文件名以 `ServiceImpl` 结尾（如 `CarsaleCarOrderServiceImpl.java`）。

这种拆分的作用：

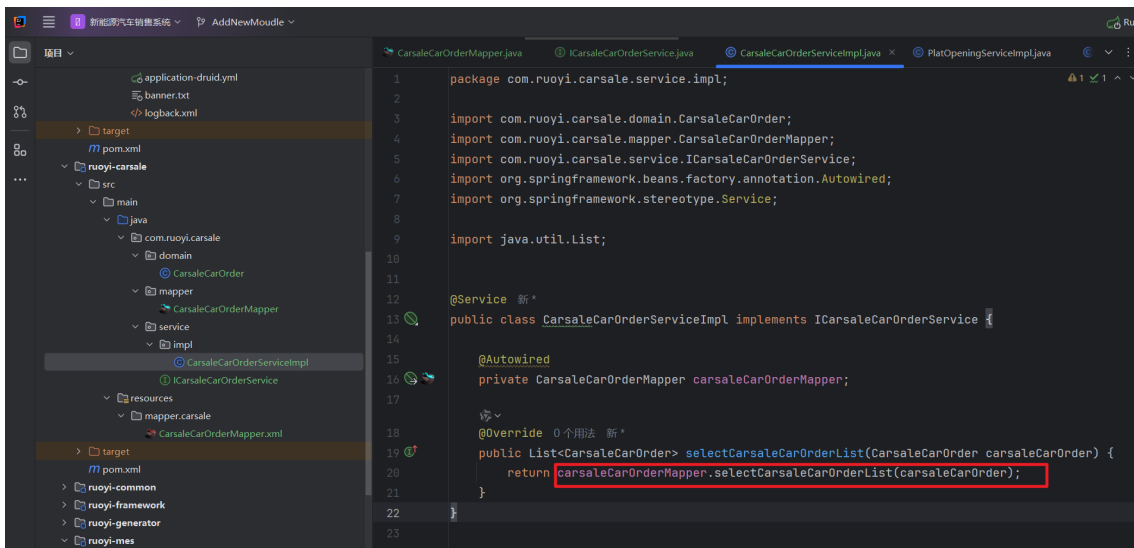
- **清晰度**：当别人想调用你的功能时，他只需要看 `service` 目录下的接口（菜单），就知道你能提供什么服务，而不需要去翻 `impl` 里成百上千行的复杂代码逻辑。
- **标准化**：这是国内主流开发（包括若依、MyBatis Plus）的标准写法。

总结你的项目结构逻辑：

1. **Controller**：负责接电话（处理请求）。
2. **Service (接口)**：负责订立合同（定义业务）。
3. **ServiceImpl (在 impl 里)**：负责干苦力活（实现业务逻辑）。
4. **Mapper**：负责找数据库拿东西（SQL 操作）。
5. **Domain**：负责装东西（数据实体）。

而在实现类【注意不是“实体类”，一个是impl、一个domain】中，写的就是具体要完成的逻辑，这一步就是具体问题具体分析了。

我这里需要的就是要调用Mapper，也就是建立个Mapper对象，然后调用其中的方法，通过Mapper映射文件，就会执行到sql语句，然后根据刚刚在xml文件里写的 `resultMap`，就会返回 `CarsaleCarOrder` 类型的Java对象

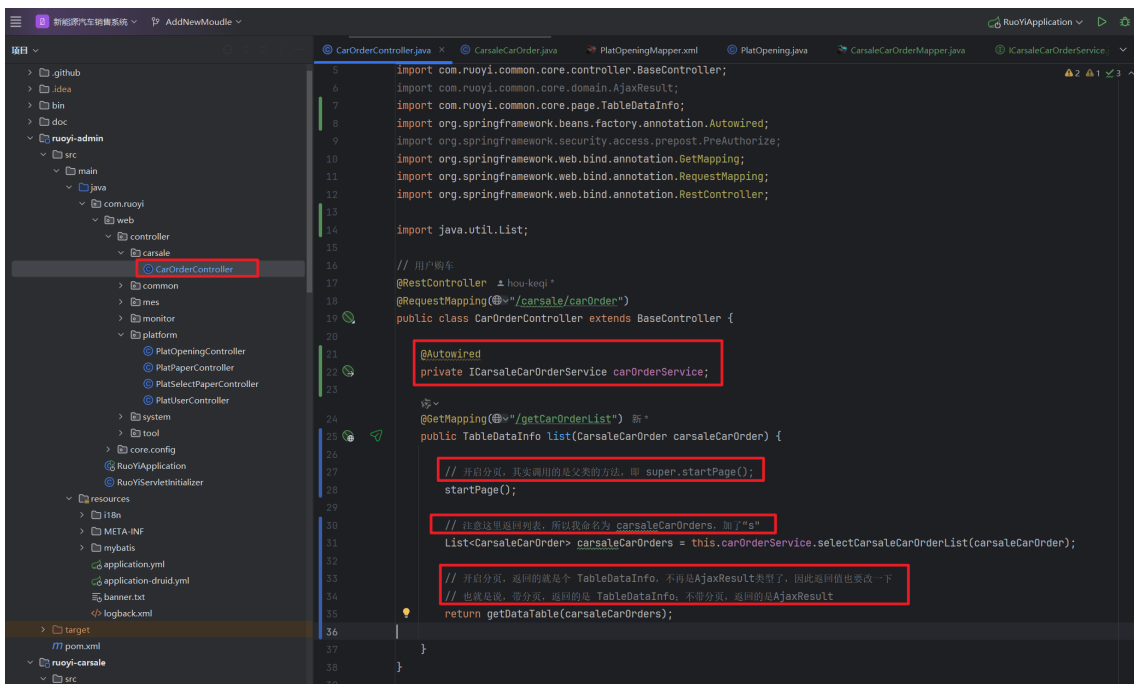


6. 那么现在情况是，Service实现类会调用Mapper，Mapper映射文件会查询数据库，给我们返回想要的东西。

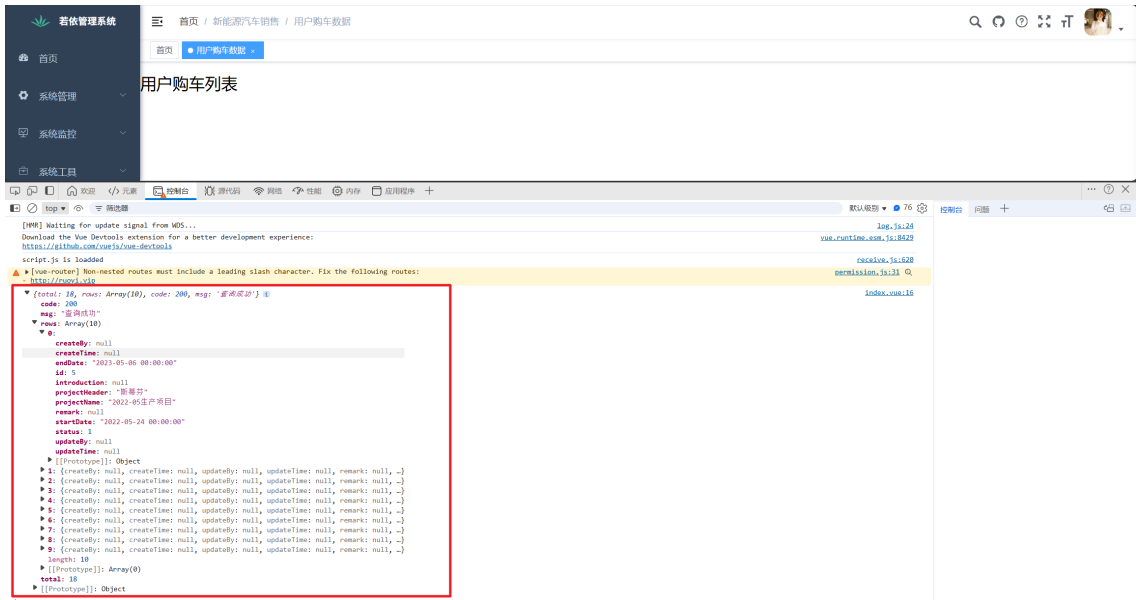
现在需要有人在前端传过来需求的时候，调用一下子Service接口，完了会自动转到Service接口对应的实现类里执行【这就是Spring注解的作用】

因此，这个案例的后端逻辑处理完了，在回去稍微修改一下昨天写的Controller里面的东西，昨天只是给他写死了，测试一下前后端通没通，现在需要在Controller层调用Service接口

注意要使用分页机制的时候，前端给出的参数里面，必须有 `pageNum`、`pageSize` 【这里插个眼，后面写前端会再次提醒】，变量名都不能变，因为若依写死的，后端就是根据这两个变量名去取参数值的。

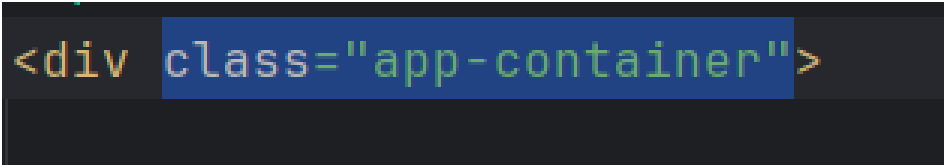
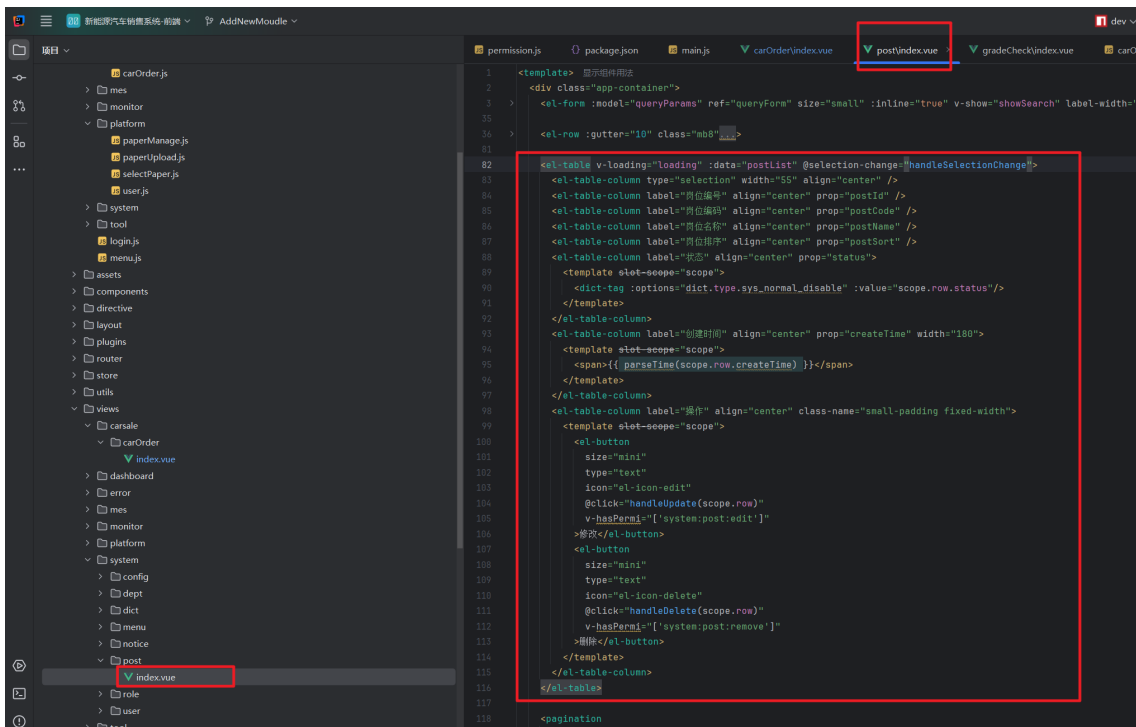


7. 运行前后端，就可以再前端看到后端传回来的数据了，剩下就交个前端了



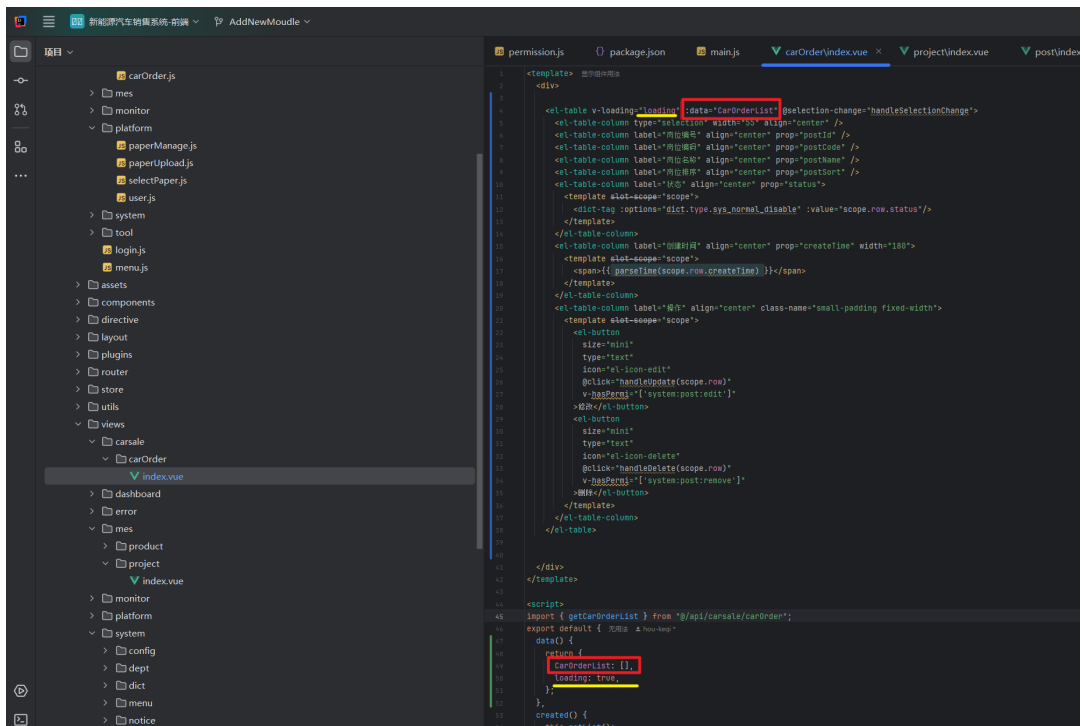
功能实现_3

1. 昨天后端已经返回数据了，今天来在前端渲染一下。这里直接把post里面的表格先抄过来，在容器这里加个样式



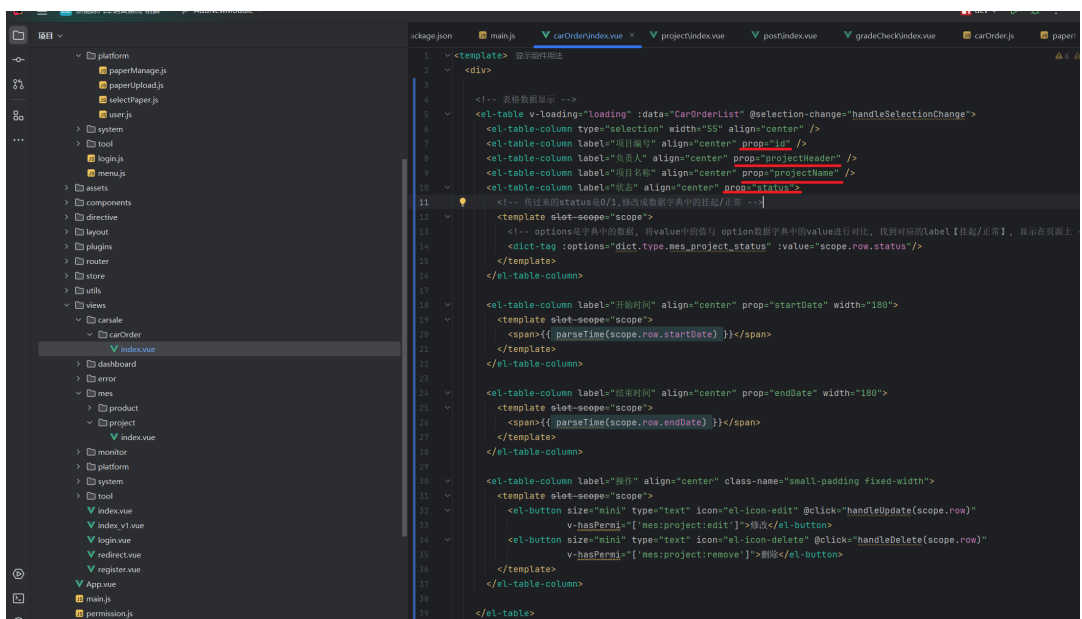
这里有几个需要注意的点：

1. 改一改里面的名词对应，添加需要的数据定义，把"岗位名词"啥的表头名词改改



- 此外，在Vue组件中，表格列的 prop 属性需要与后端返回的JSON数据中的属性名保持一致【而不是数据库字段名】，而我们后端数据库查询后，经过结果映射，返回的就是 CarsaleCarOrder类型的Java对象

也就是说，这里 prop属性名 与 后端实体类里面各个属性名 是一对一绑定的



- 这里后端返回的状态值（status）是 0 或 1，0代表挂起，1代表正常

传过来的status是0/1，我们想让前端显示的是挂起/正常，就可以借助若依项目中的字典，自动实现。

若依管理系统

首页

系统管理

用户管理

角色管理

菜单管理

部门管理

岗位管理

字典管理

参数设置

通知公告

字典管理

字典名称 请输入字典名称

字典类型 请输入字典类型

搜索重置

新增修改删除导出刷新缓存

	字典编号	字典名称	字典类
☐	1	用户性别	sys_user
☐	2	菜单状态	sys_show
☐	3	系统开关	sys_normal_
☐	4	任务状态	sys_job_s
☐	5	任务分组	sys_job_g
☐	6	系统是否	sys_yes

修改字典类型

* 字典名称

用户购车状态

* 字典类型

carsale_carorder_status

状态

正常

停用

备注

新能源汽车销售系统

确定

取消

在点进去新建出来字典，新增对应关系

若依管理系统

首页

系统管理

用户管理

角色管理

菜单管理

部门管理

岗位管理

字典管理

参数设置

通知公告

字典管理

字典名称 请输入字典名称

字典类型 请输入字典类型

状态 字典状态

创建时间 开始日期 结束日期

搜索重置

新增修改删除导出刷新缓存

	字典编号	字典名称	字典类型	状态	备注	创建时间	操作
☐	100	生产状态	mes_product_status	正常		2024-06-29 09:59:20	修改删除
☐	101	项目状态	mes_project_status	正常		2024-07-01 10:49:18	修改删除
☐	102	用户身份	platform_user_role	正常	论文管理平台的用户身份	2024-07-03 08:57:56	修改删除
☐	104	用户购车状态	carsale_carorder_status	正常	新能源汽车销售系统	2026-01-17 21:47:37	修改删除

共 14 条 10条/页 1 2 前往 2 页

添加字典数据

×

字典类型

carsale_carorder_status

* 数据标签

挂起

* 数据键值

0

样式属性

请输入样式属性

* 显示排序

0

^
v

回显样式

默认(default)

v

状态

☒

正常

☐

停用

备注

请输入内容

确定

取消

修改字典数据

×

字典类型

carsale_carorder_status

* 数据标签

正常

* 数据键值

1

样式属性

请输入样式属性

* 显示排序

1

^
v

回显样式

默认(default)

v

状态

☒

正常

☐

停用

备注

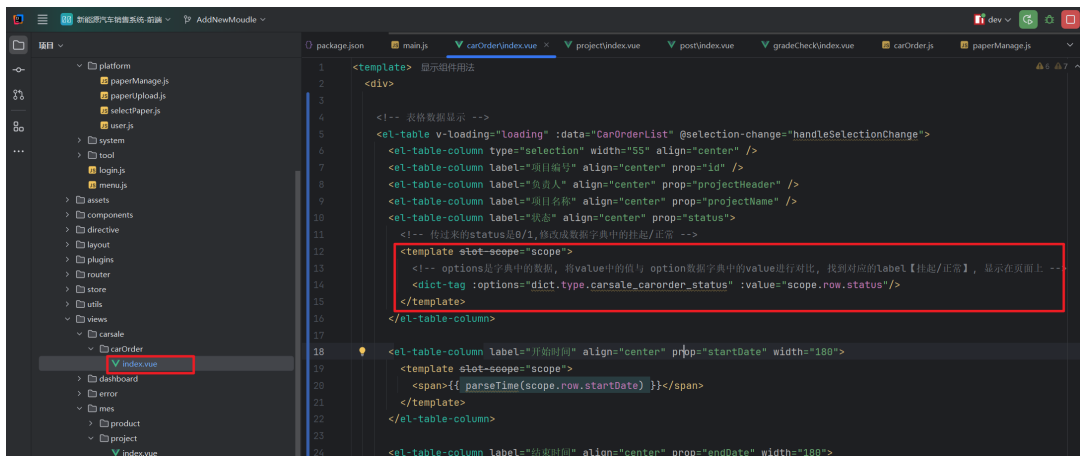
请输入内容

确定

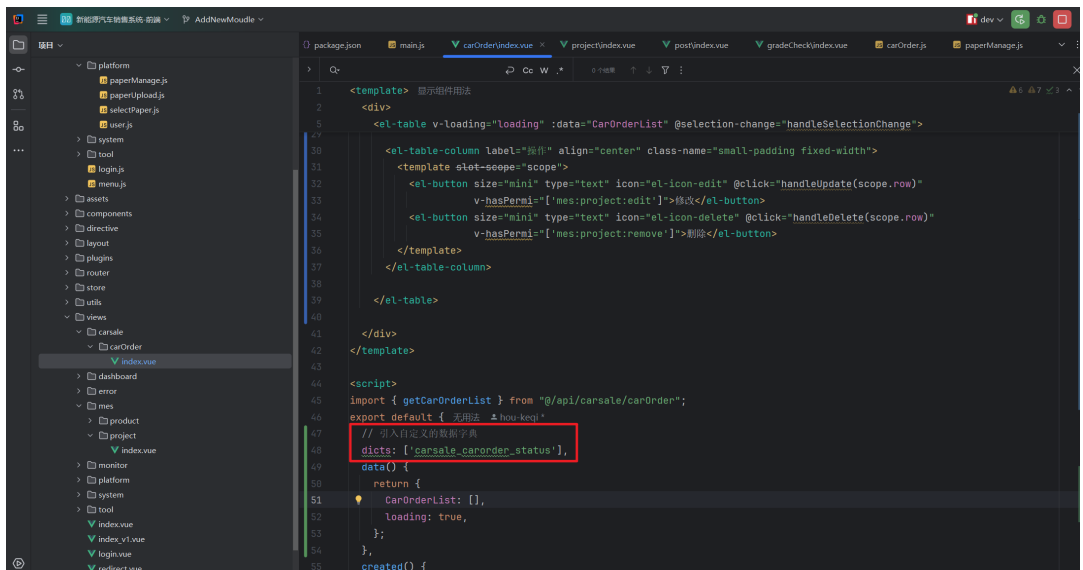
取消

建立还数据字典之后，还要在前端写自定义的标签用来渲染

下图大红框中代码解释是：在表格列中显示项目状态的标签。通过slot-scope获取当前行数据，使用dict-tag组件将状态值（status）与字典数据（carsale_carorder_status）进行匹配，将数字状态值转换为对应的中文标签（如"挂起/正常"）展示在页面上。



此外，以为这个字典是自定义出来的，因此要在下面额外引入一下

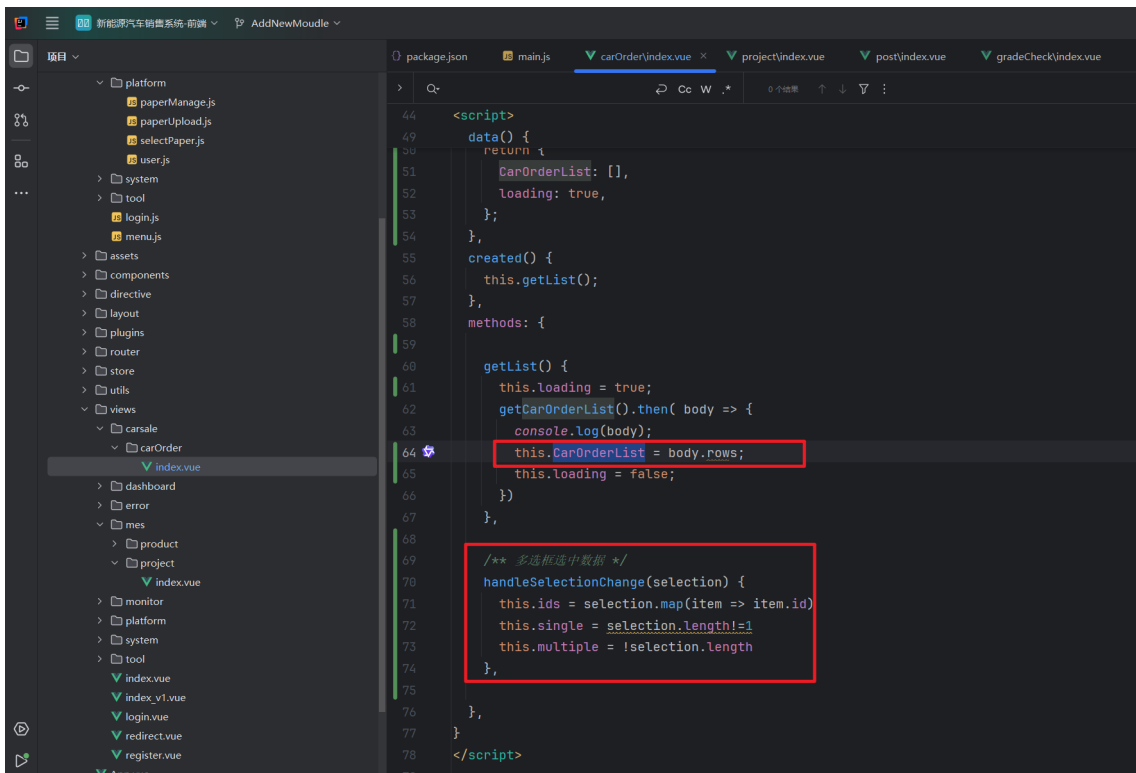


此外，在"论文管理平台"项目中，后端返回来的角色信息是以"ROLE_ADMIN"、"ROLE_TEACHER"、"ROLE_STUDENT"存储返回的，不过在前端就可以简单在字典管理里面完成关联，就可以显示"管理员"、"教师"、"学生"，可供参考借鉴

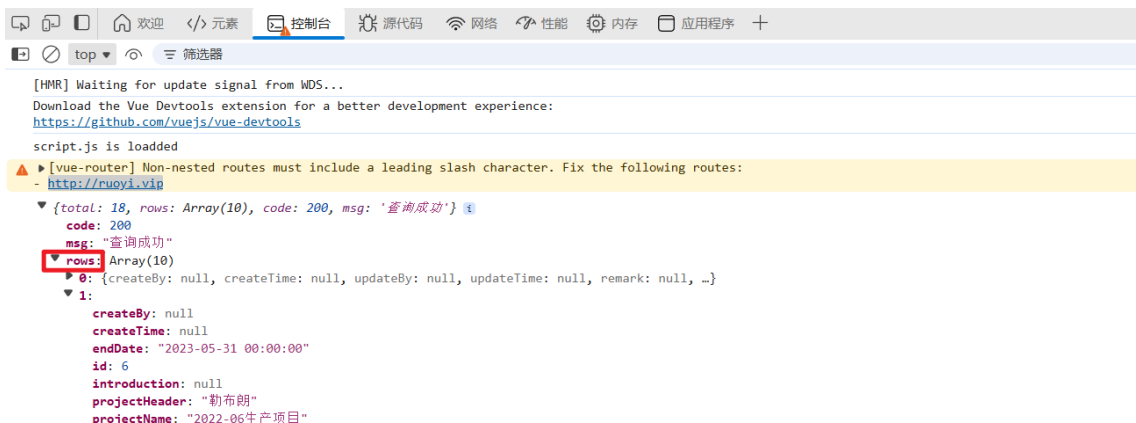
	字典编码	字典标签	字典键值	字典排序	状态	备注	创建时间	操作
☐	103	管理员	ROLE_ADMIN	1	正常	ROLE_ADMIN = 管理员	2024-07-03 09:03:18	去修改 删除
☐	102	教师	ROLE_TEACHER	2	正常	ROLE_TEACHER = ...	2024-07-03 09:02:38	去修改 删除
☐	104	学生	ROLE_STUDENT	3	正常	ROLE_STUDENT = ...	2024-07-03 09:04:14	去修改 删除

共 3 条 10条/页 4 前往 1 页

2. 前端渲染报错，两个原因 ①没给出 handleSelectionChange 方法，多选框的东西，在mes里面抄过来 ②忘记把后端返回来的数据放到 carorderList 里面了



注意我们这里给 CarOrderList 赋值的时候，是body里面的rows，不是data 属性，这点从前端控制套可以看出

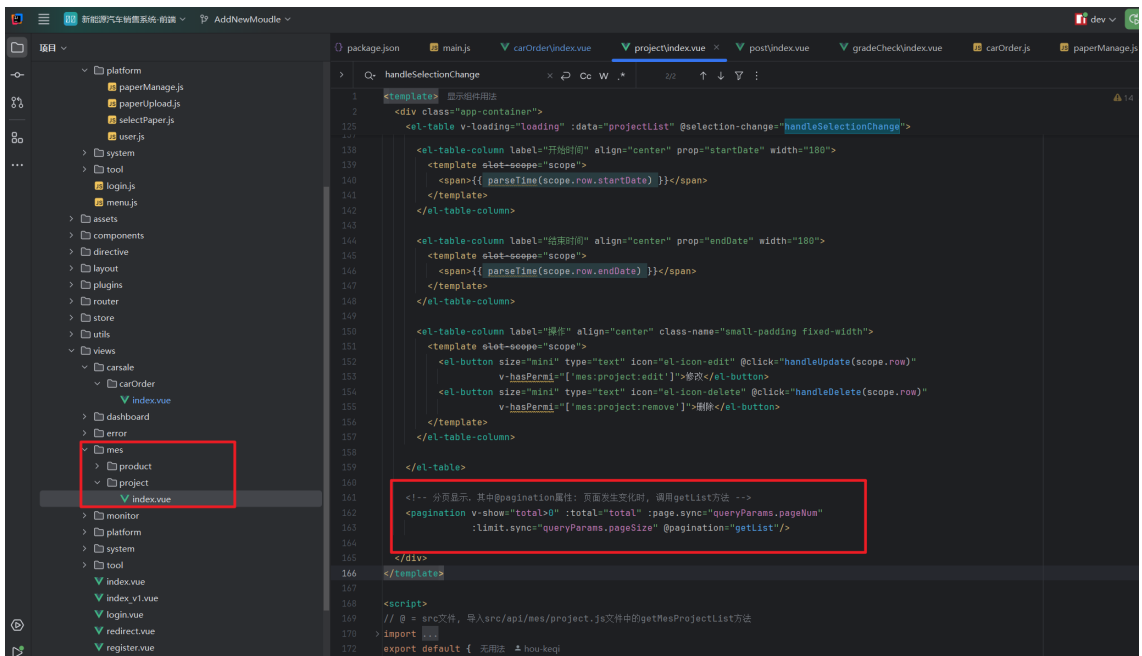


3. 刷新后，我们就可以看到数据了



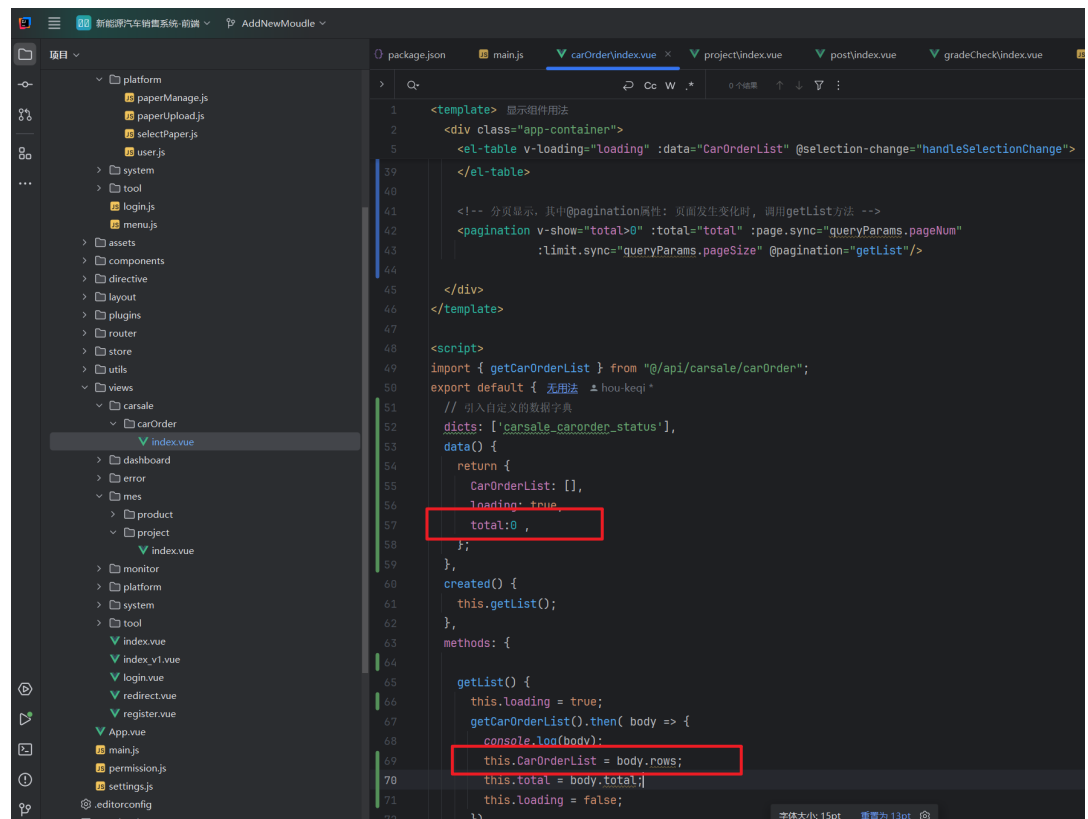
4. 接下来，我们要做的是"分页"和"搜索框"

首先，把分页栏前段代码抄过来，目前这个功能完全仿照mes业务模块的project功能，直接抄

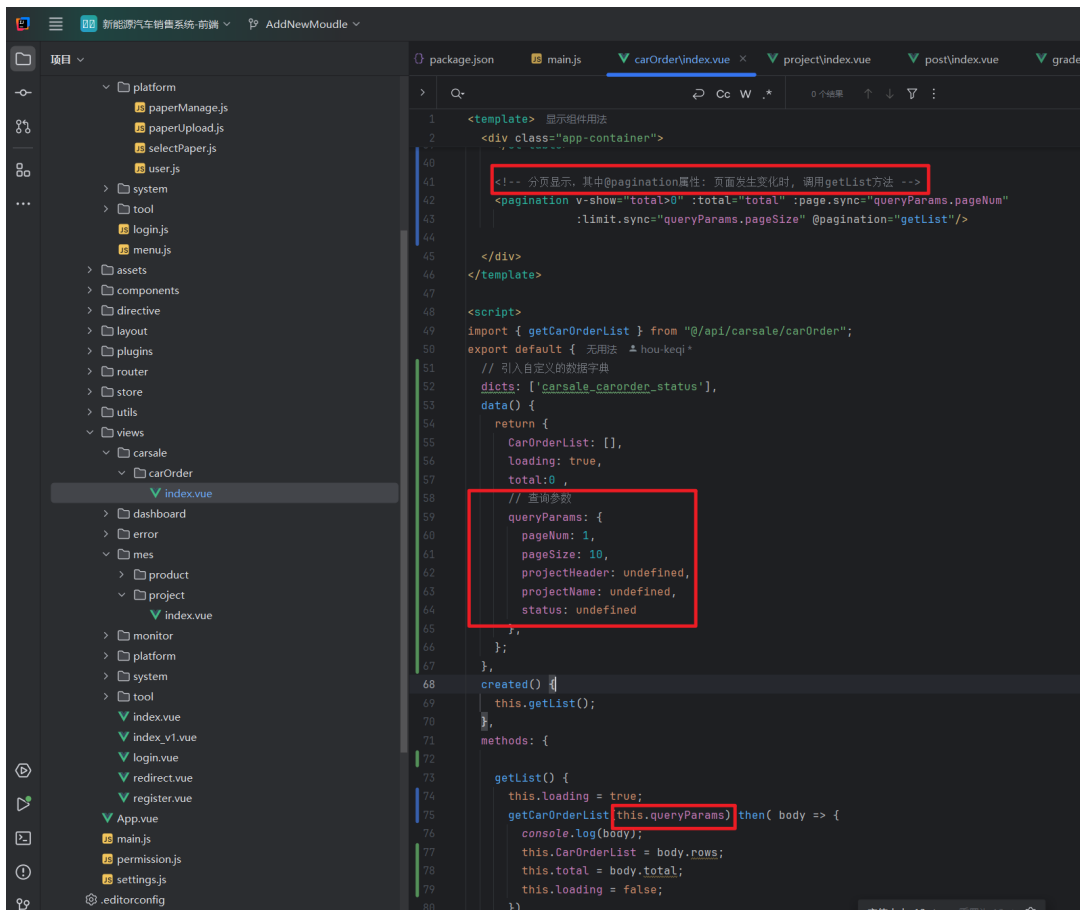


这里要注意

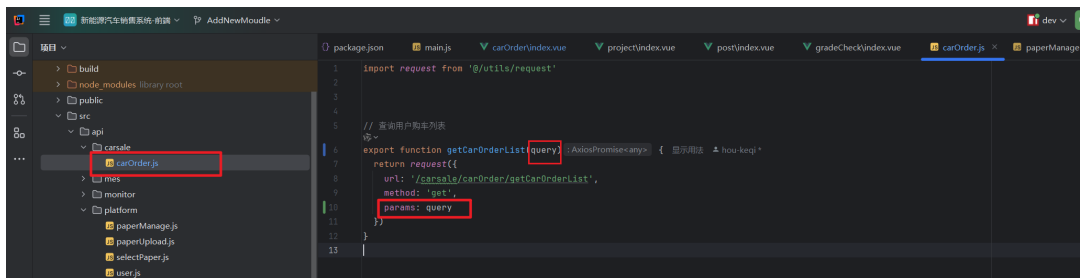
1. 增加total的属性定义、在后端返回来的时候给total赋值



2. 前端查询的参数内容，会封装到 queryParams 中，因此要增加 queryParams 的属性定义，其中就包括前面【插眼1的地方】强调的 pageSize、pageNum，查第2页、第3页的时候，这里相当于是把各种信息放到 queryParams 里，从前端把查询内容传给后端，因此要在 getCarOrderList 里面把参数带上



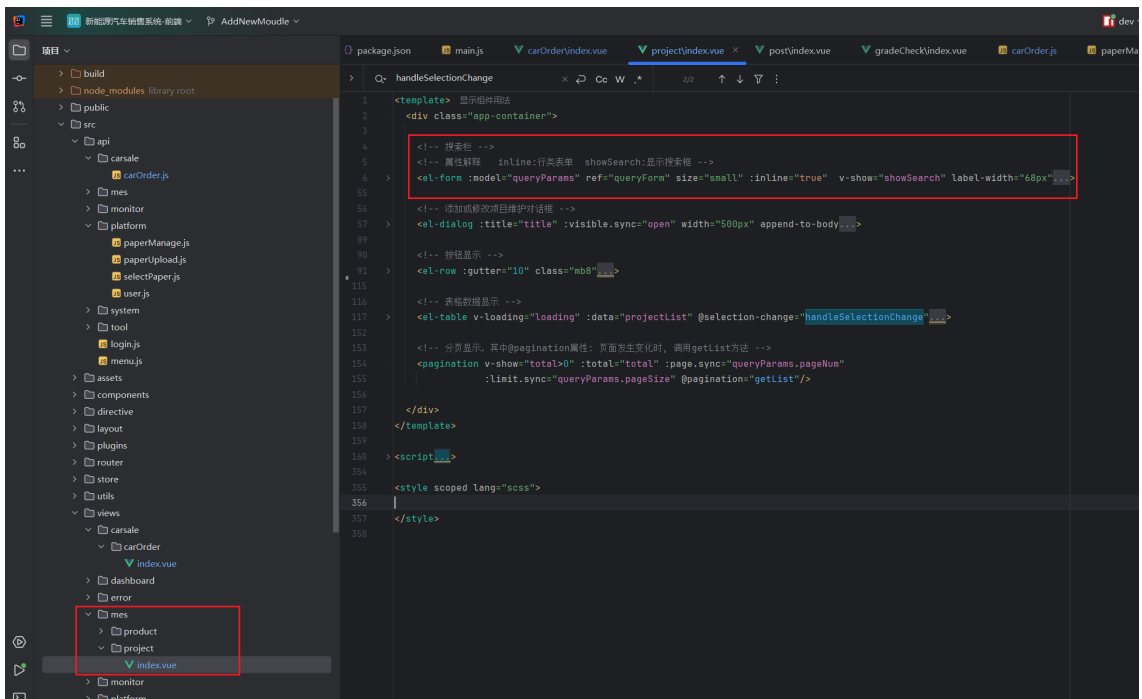
3. 那么现在问题时，当时咱们定义的调用后端接口的 `getCarOrderList` 里面，没有带参数，现在就需要带参数了。因此要改一下api调用，在api目录里修改



这样，前端就完成分页了

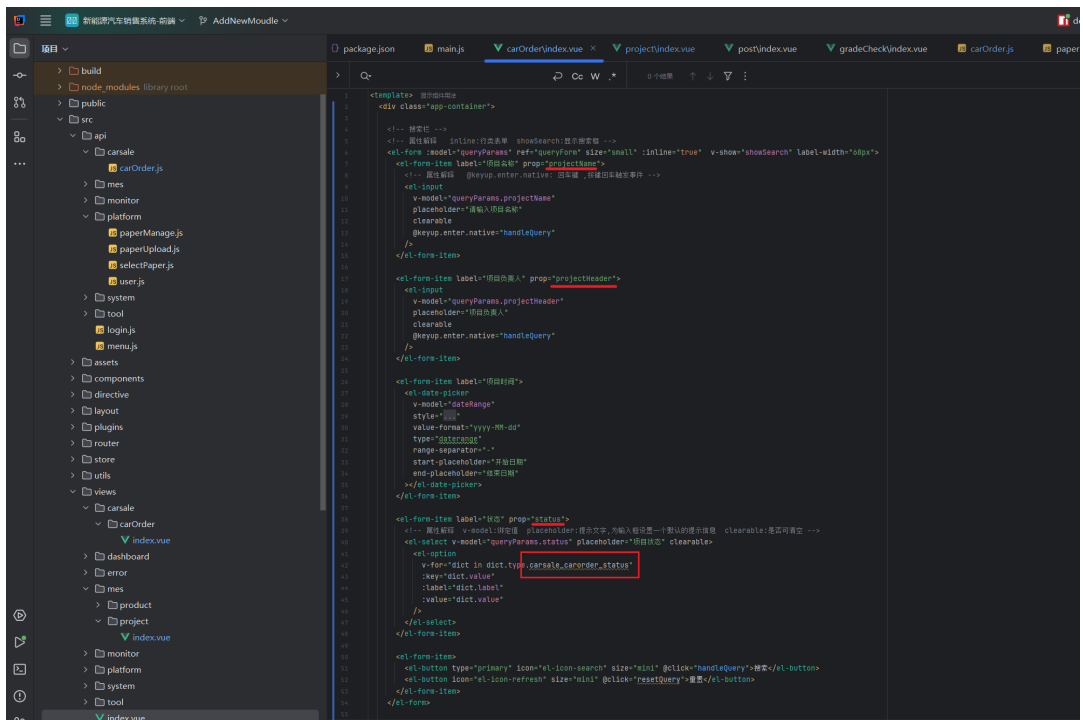


5. 接下来继续完成"搜索栏"功能，先把mes里面的搜索框抄过来



注意加一些东西

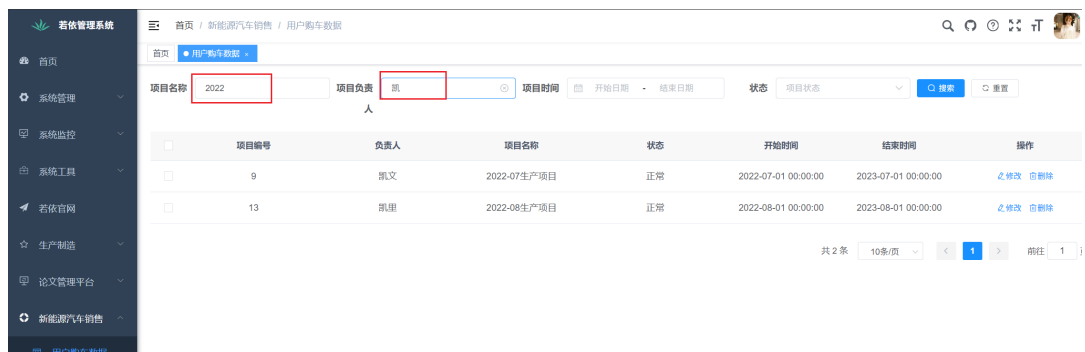
1. 把抄过来的代码改改，因为我现在是举例，用的都是mes人家的数据库，所以改的少，到时候你们数据库弄出来，这些东西名字都要改对



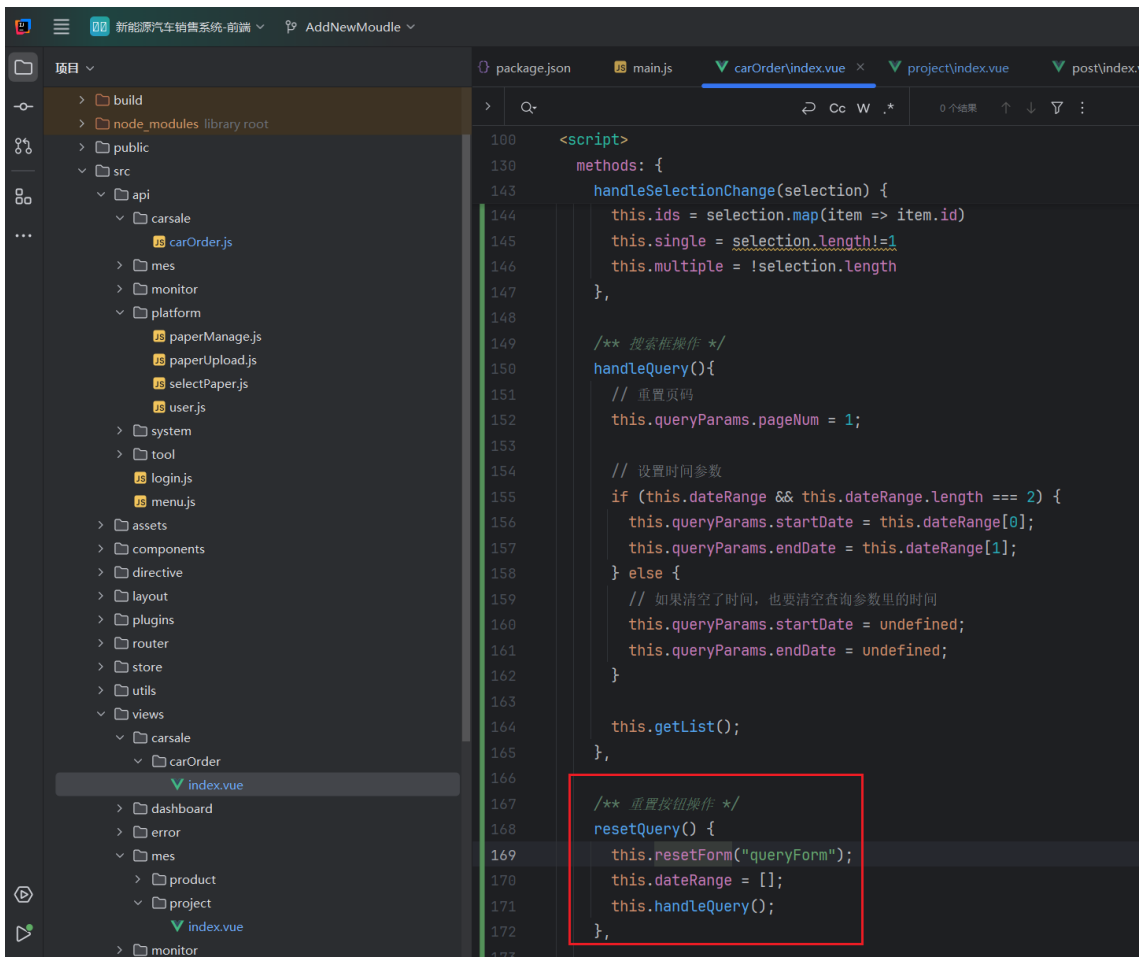
2. 加一些搜索框需要的属性定义和方法，注释里面都写解释了

```
104 <script>
105 // 引入自定义的封装函数
106 dict: ['pachale-garorder-status'],
107 data() {
108   return {
109     CarOrderList: [],
110     loading: true,
111     total: 0
112   }
113 }
114 // 查询参数
115 queryParams: {projectHeader: undefined...},
116 // 是否显示搜索栏
117 showSearch: true,
118 // 时间范围: dateRange[0]代表开始时间, dateRange[1]代表结束时间, 具体赋值的时候, 会在 handleQuery 函数里完成
119 dateRange: [],
120 };
121 created() {
122   this.getList();
123 },
124 methods: {
125
126   getList() {
127     this.loading = true;
128     getCarOrderList(this.queryParams).then( body => {
129       console.log(body);
130       this.CarOrderList = body.rows;
131       this.total = body.total;
132       this.loading = false;
133     })
134   },
135
136   /** 多选框选中数据 */
137   handleSelectionChange(selection) {
138     this.id = selection.map(item => item.id)
139     this.single = selection.length===1
140     this.multiple = !selection.length
141   },
142
143   handleQuery(){
144     // 重置页码
145     this.queryParams.pageNum = 1;
146
147     // 设置时间范围
148     if (this.dateRange && this.dateRange.length === 2) {
149       this.queryParams.startDate = this.dateRange[0];
150       this.queryParams.endDate = this.dateRange[1];
151     } else {
152       // 如果清空了时间, 也要清空查询数据的时间
153       this.queryParams.startDate = undefined;
154       this.queryParams.endDate = undefined;
155     }
156   }
157 }
158 }
159 }
160 }
161 }
162 }
```

这样就可以完成查询了



6. 这里控制套会有些报错, 那是因为我们还没搞重置功能, 现在来完成一下, 就是在点击重置按钮的时候, 把表单所有参数重新初始化, 然后再执行一遍查询操作就完了



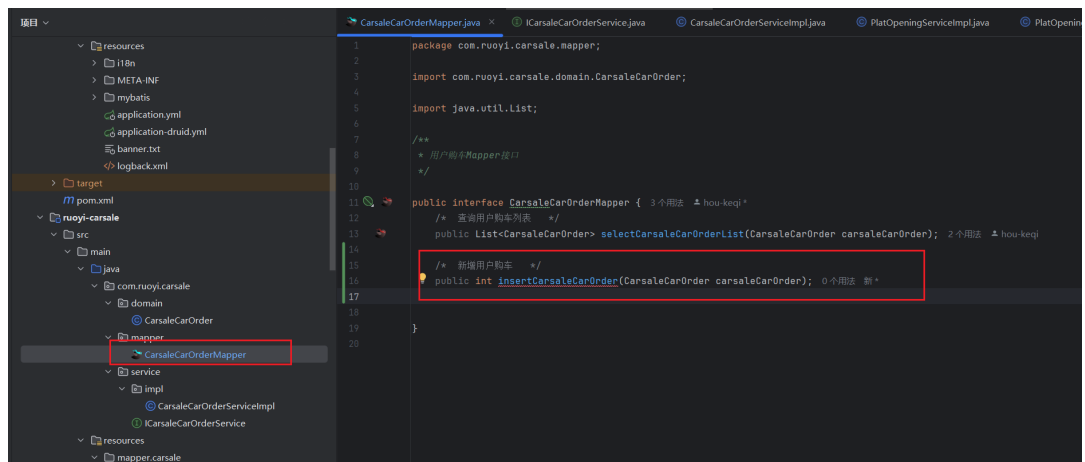
至此，“搜索功能”和“分页功能”就完成了。

功能实现_4

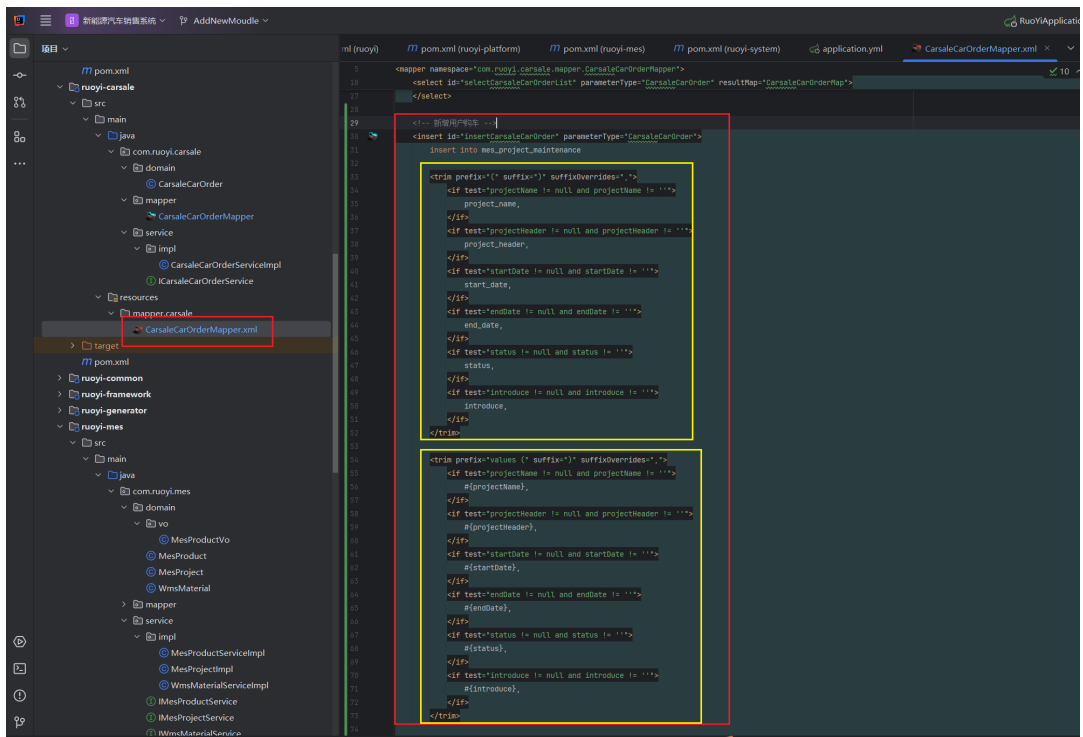
今天来完成“用户购车”的增删改功能

1. 现在后端把处理逻辑加上，首先写Mapper接口和他的实现

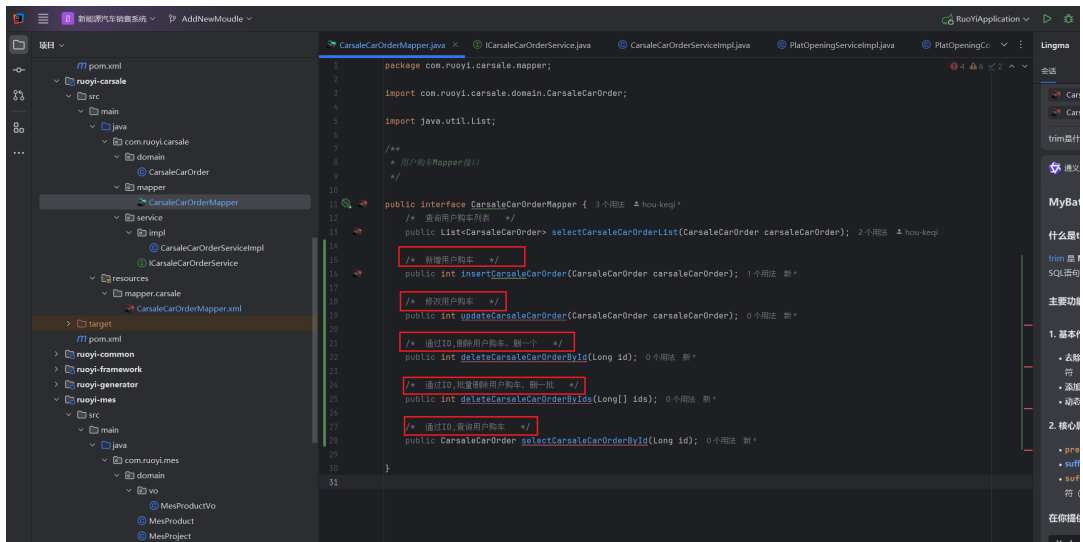
1. 添加Mapper新增功能的接口



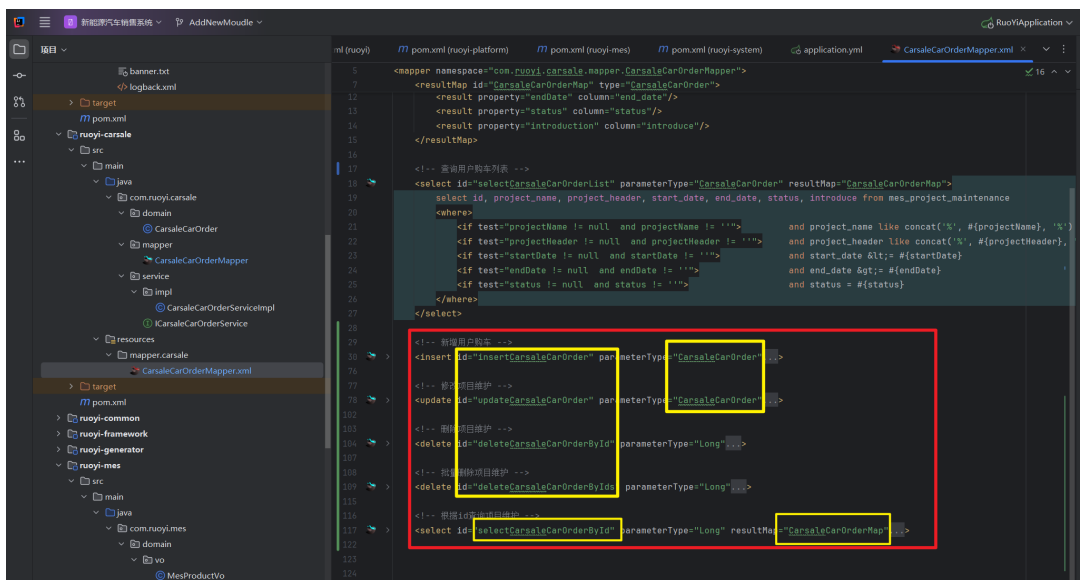
2. 在Mapper映射文件里面添加新增功能相应的SQL语句，可以查一下标签的作用，拼接成SQL语句，依旧注意id是Mapper接口的函数名 `insertCarsaleCarOrder`，参数类型依旧是 `CarsaleCarOrder` 实体类



3. 同样操作，先加Mapper功能接口

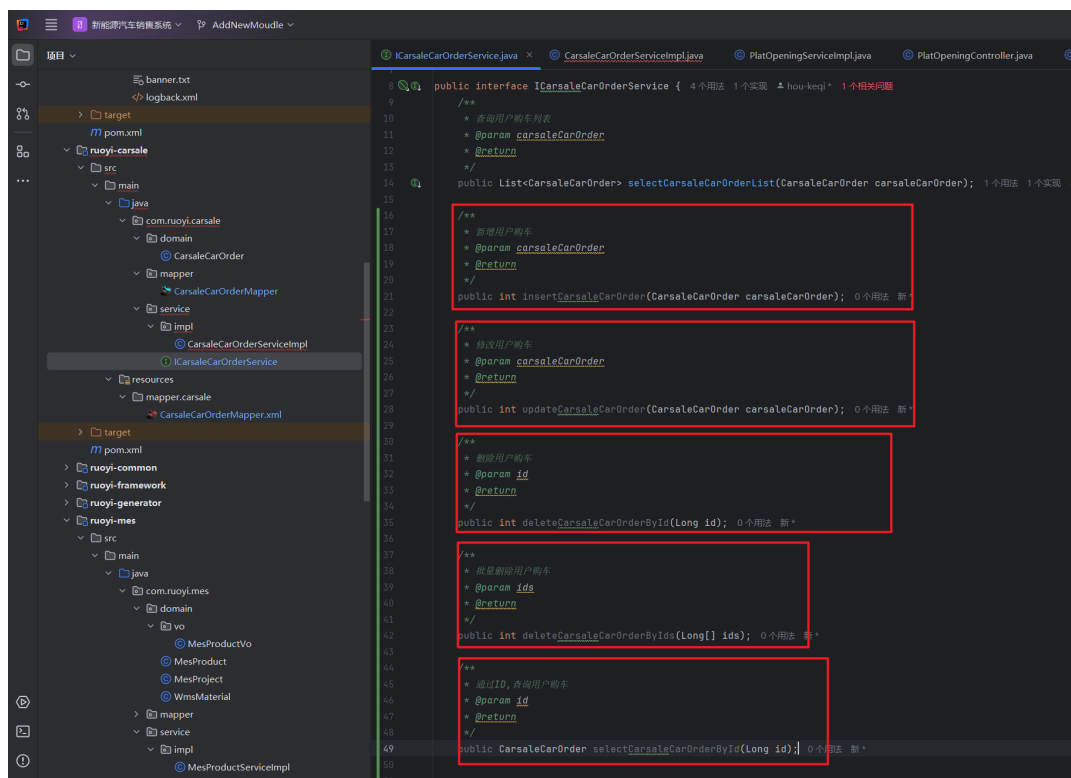


再在Mapper映射文件中写对应的SQL语句，我直接抄的mes模块里的，不过改一下id和参数类型



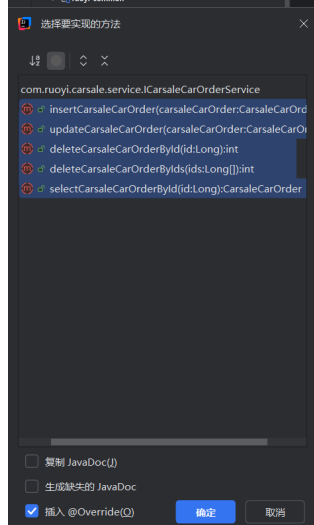
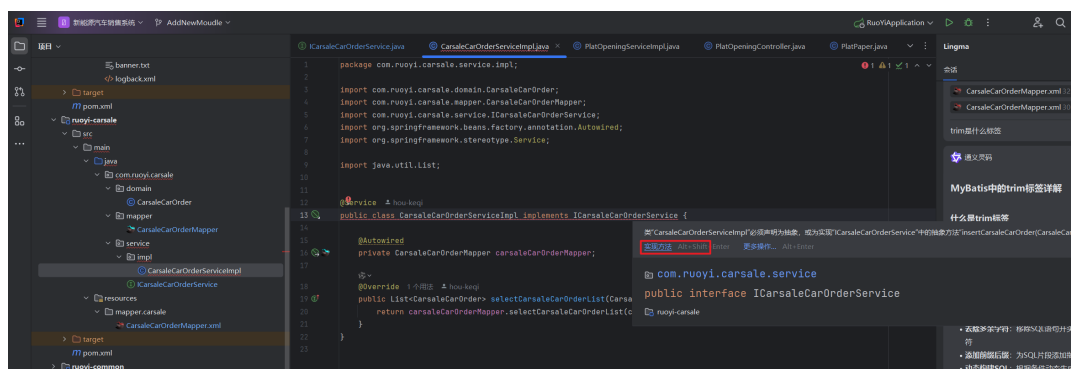
2. 接下来，该写Service接口和他的实现类了

1. 先写Service接口，通义灵码基本上一路 Tab 和 回车 就行

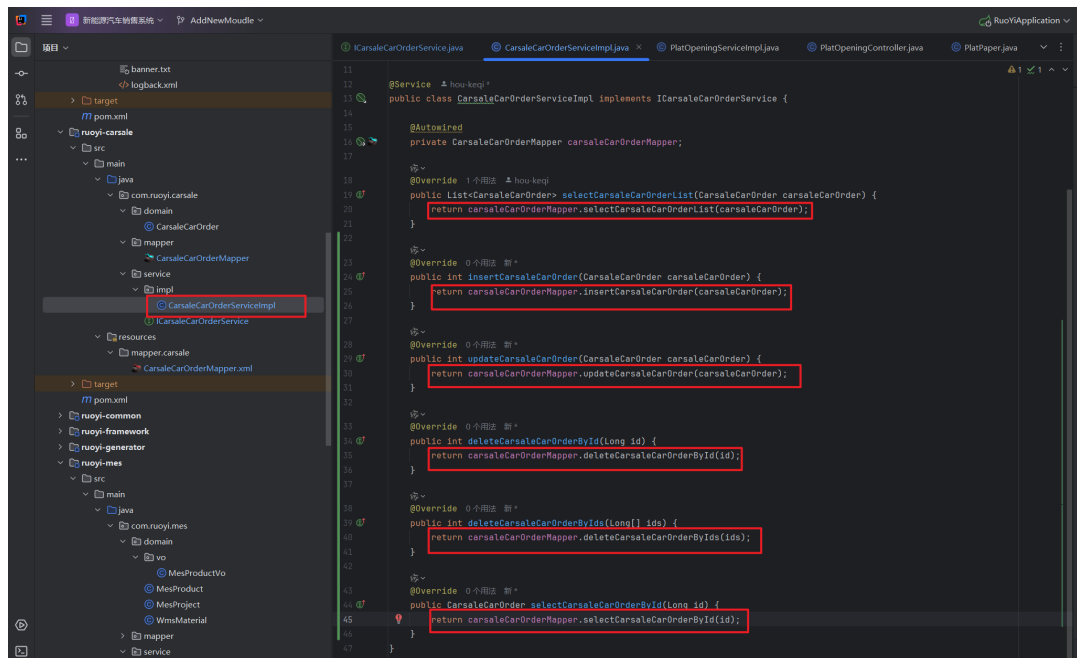


2. 完成Service实现类

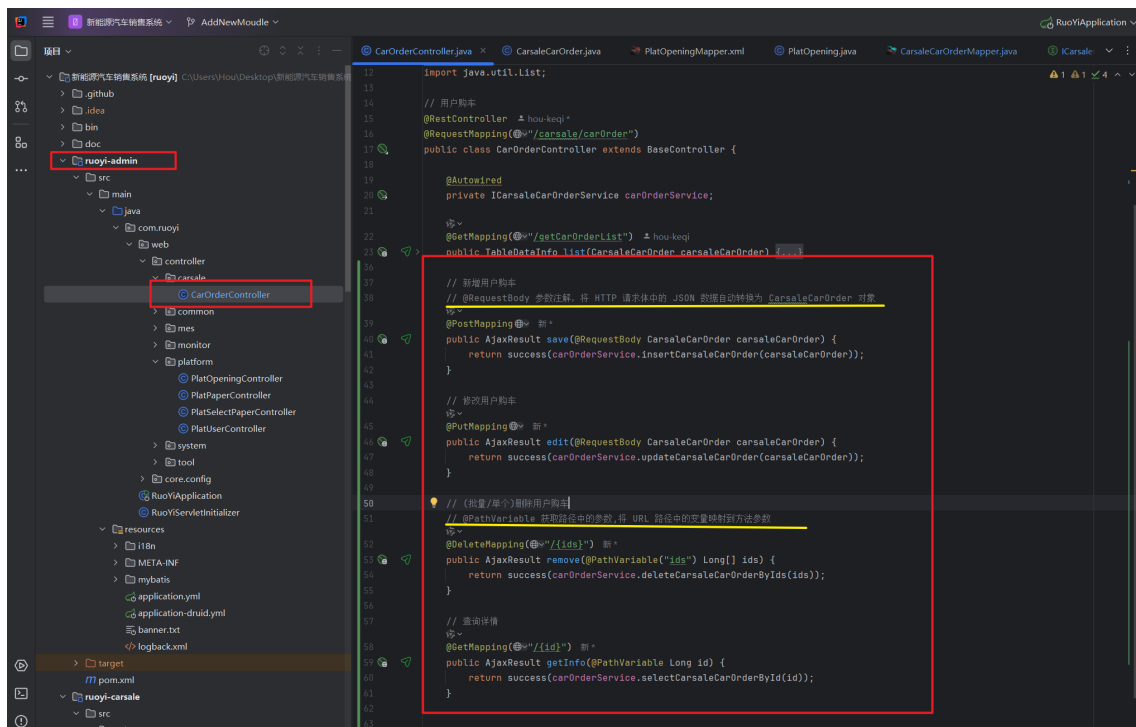
直接先生成



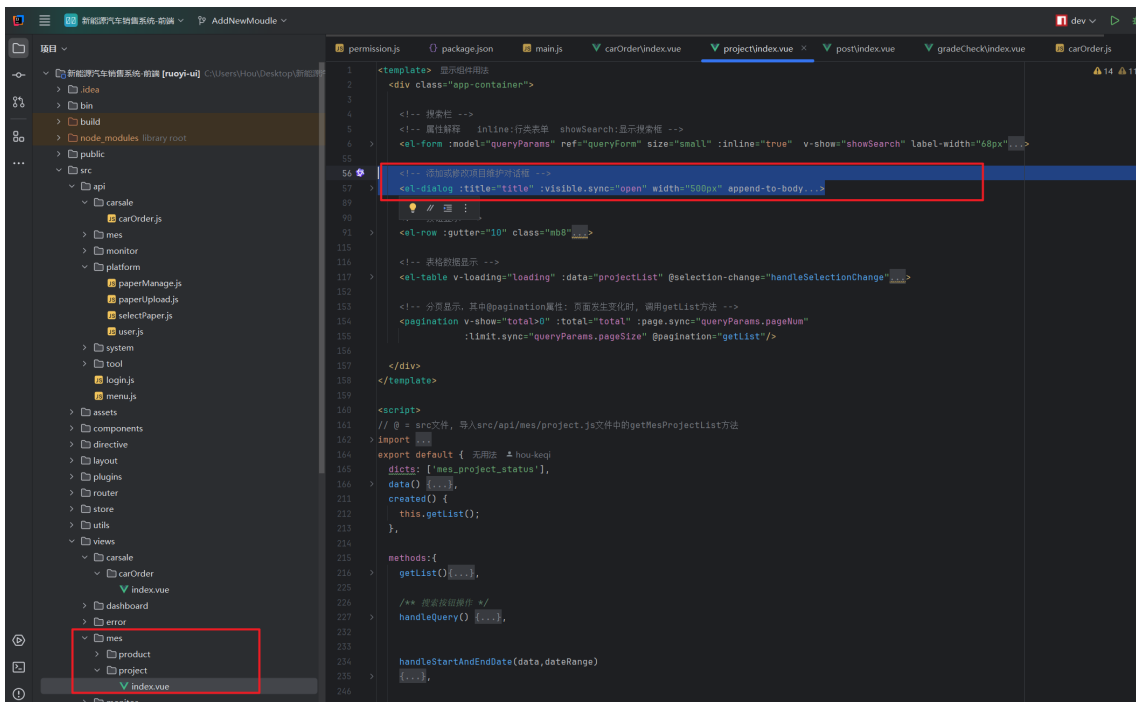
然后再去调用Mapper



3. 最后写Controller

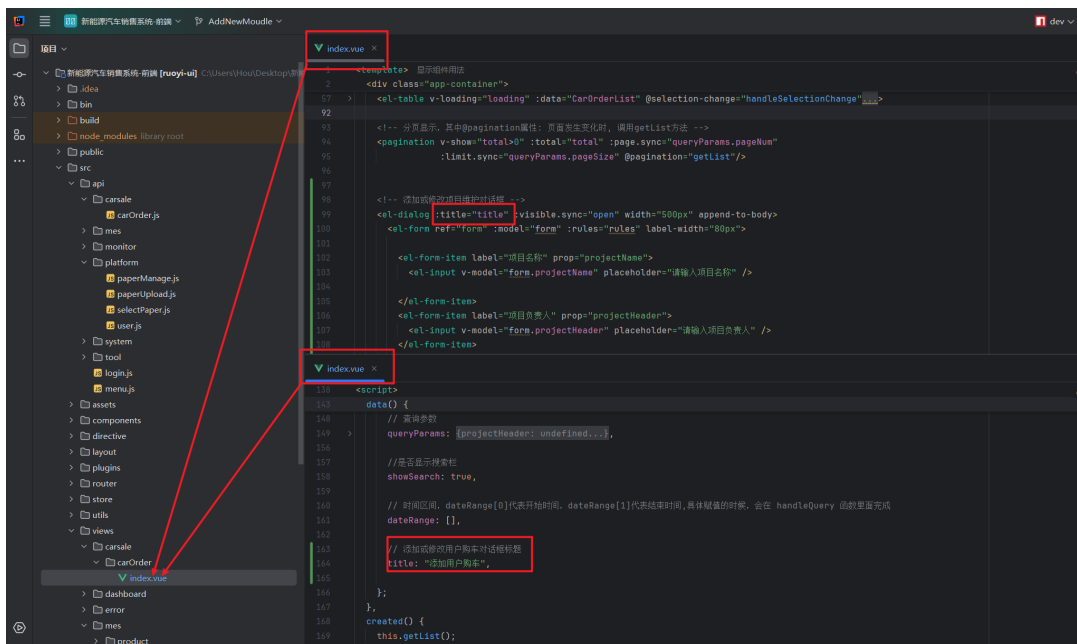


4. 后端写完了，接着写前端，直接抄mes的，粘贴到carsale里面，改改名字

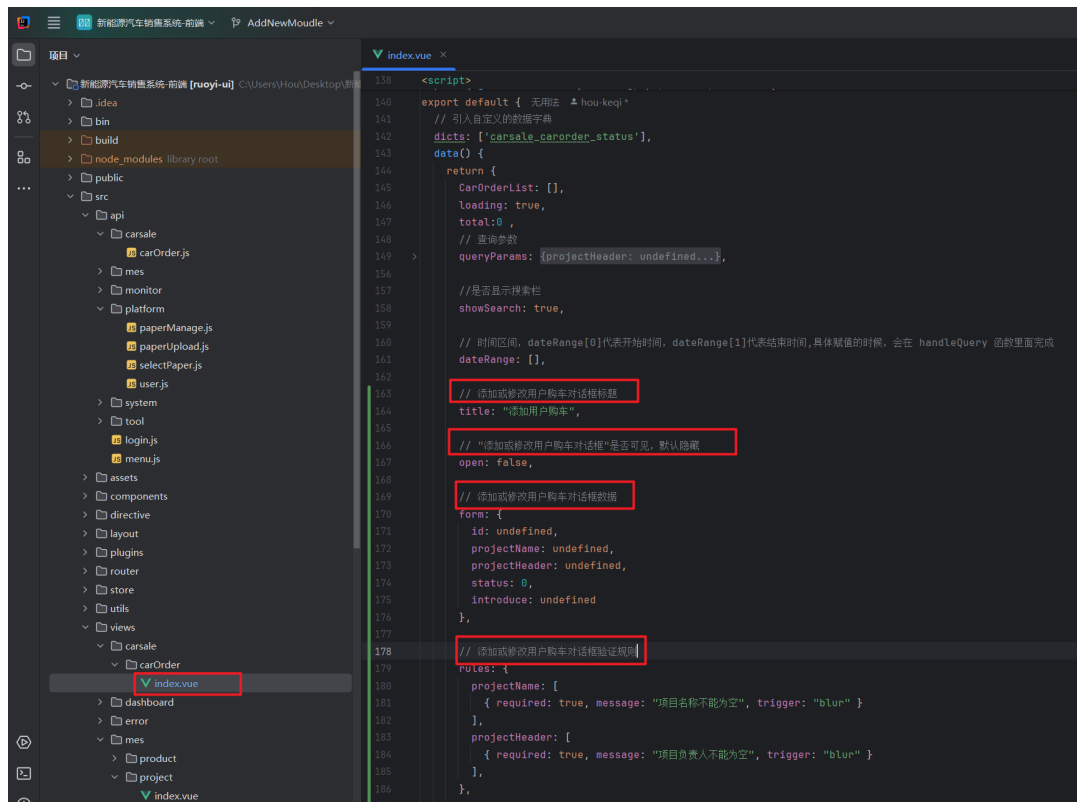


5. 同样，这里有几个注意的事情

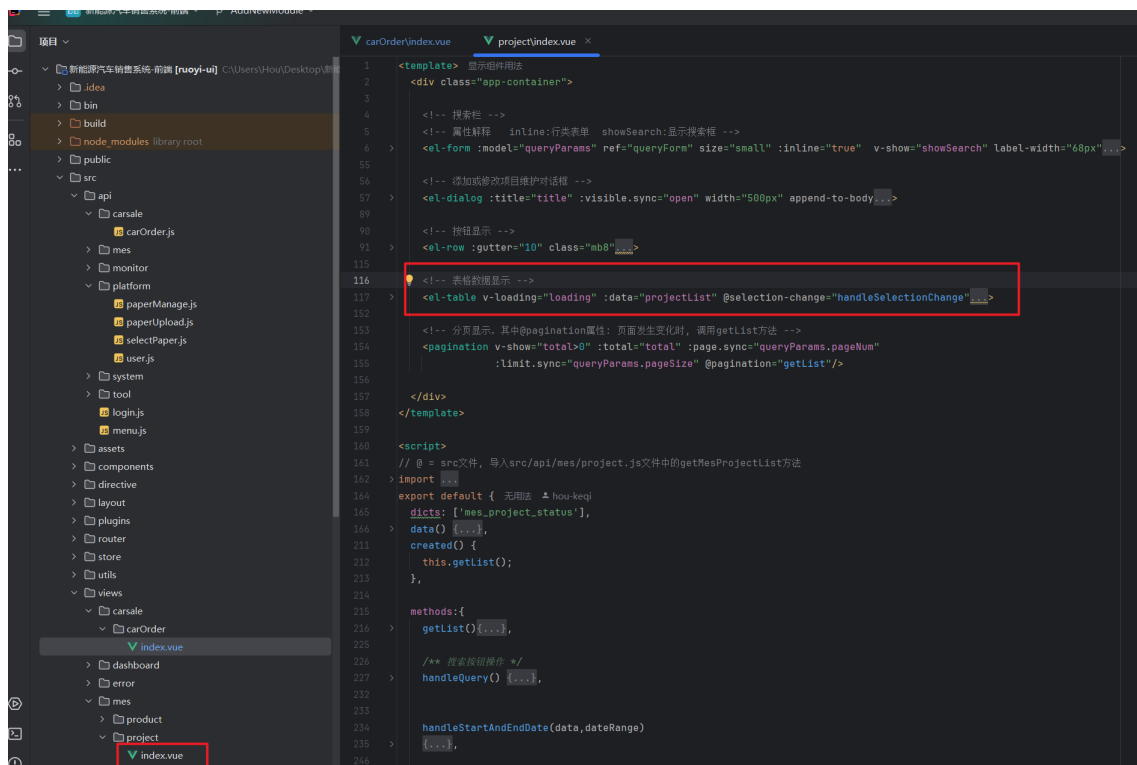
1. 由于新增和修改的对话框，这里用的是同一个，因此用一个“title”字段，显示标题作区分就行



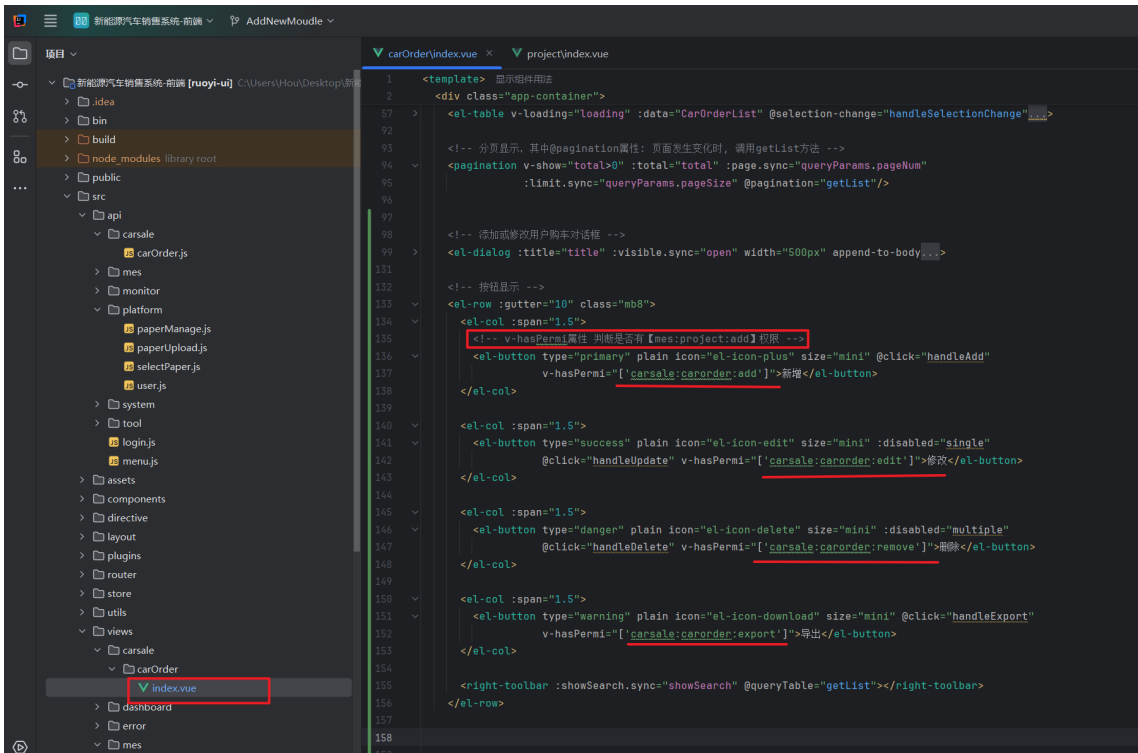
2. 补一些对话框需要的属性定义，记得把上面的状态字典值改一下



6. 下面添加按钮，依旧抄

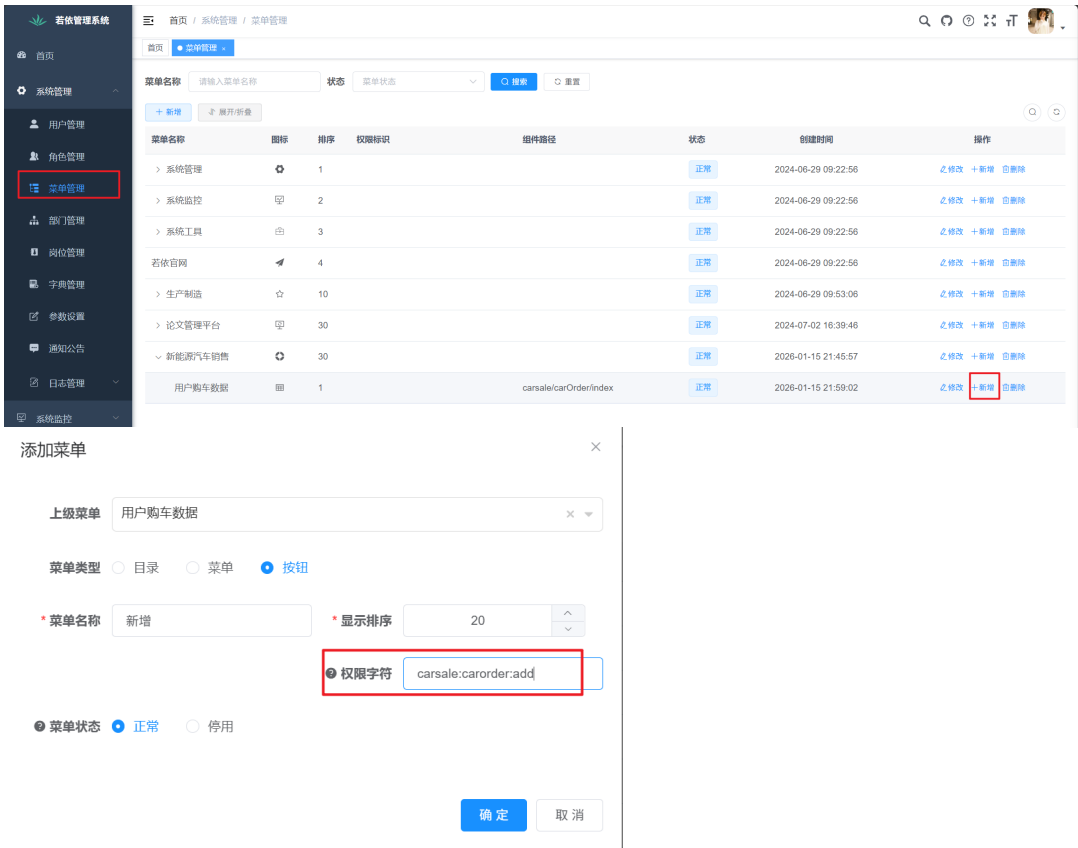


7. 这里肯定不是任何角色都能删除新增导出的，要有权限验证

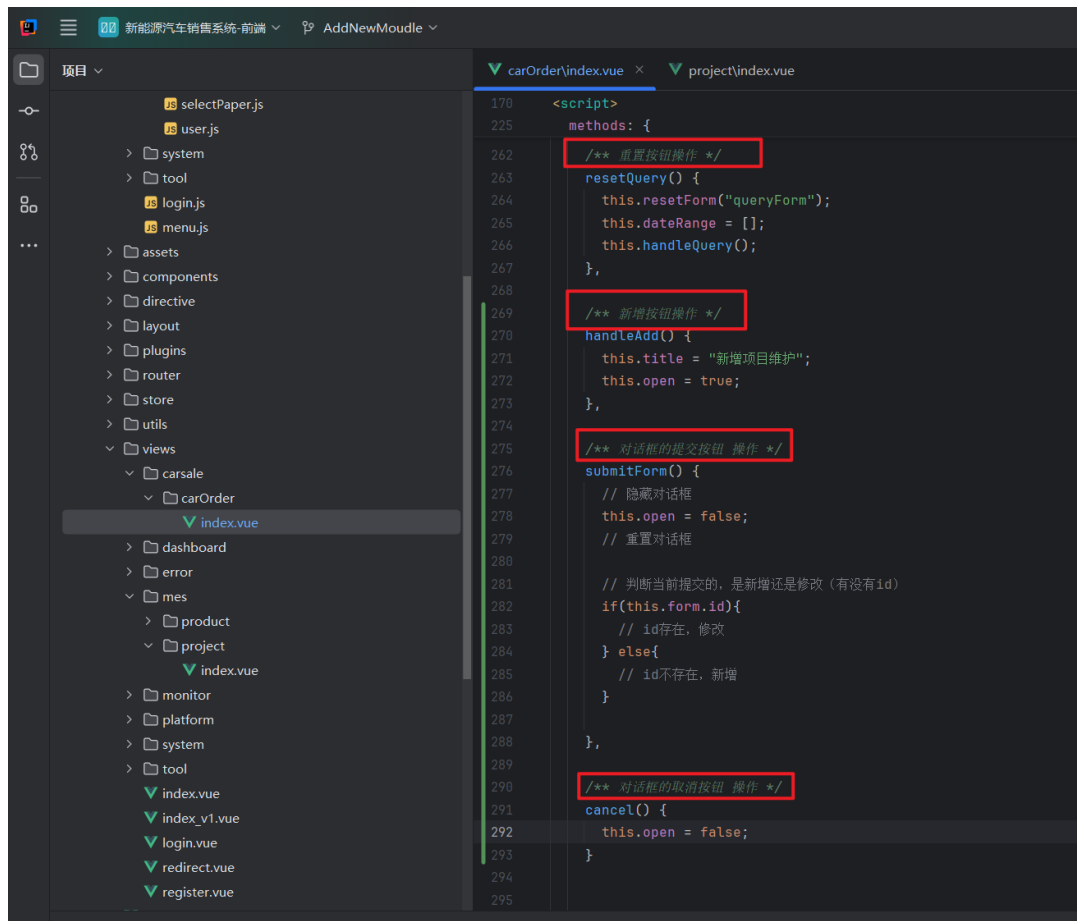


这个权限当然是要在UI界面添加，更新到数据库里面的，这里以“新增”按钮为例，其他同理

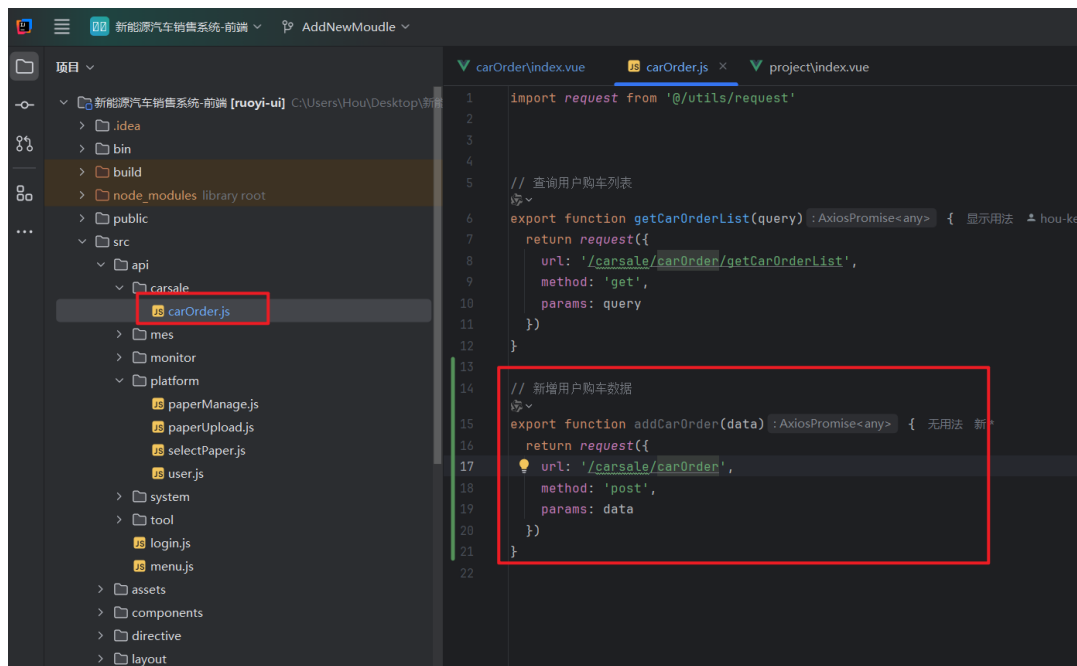
1. 添加按钮所需权限



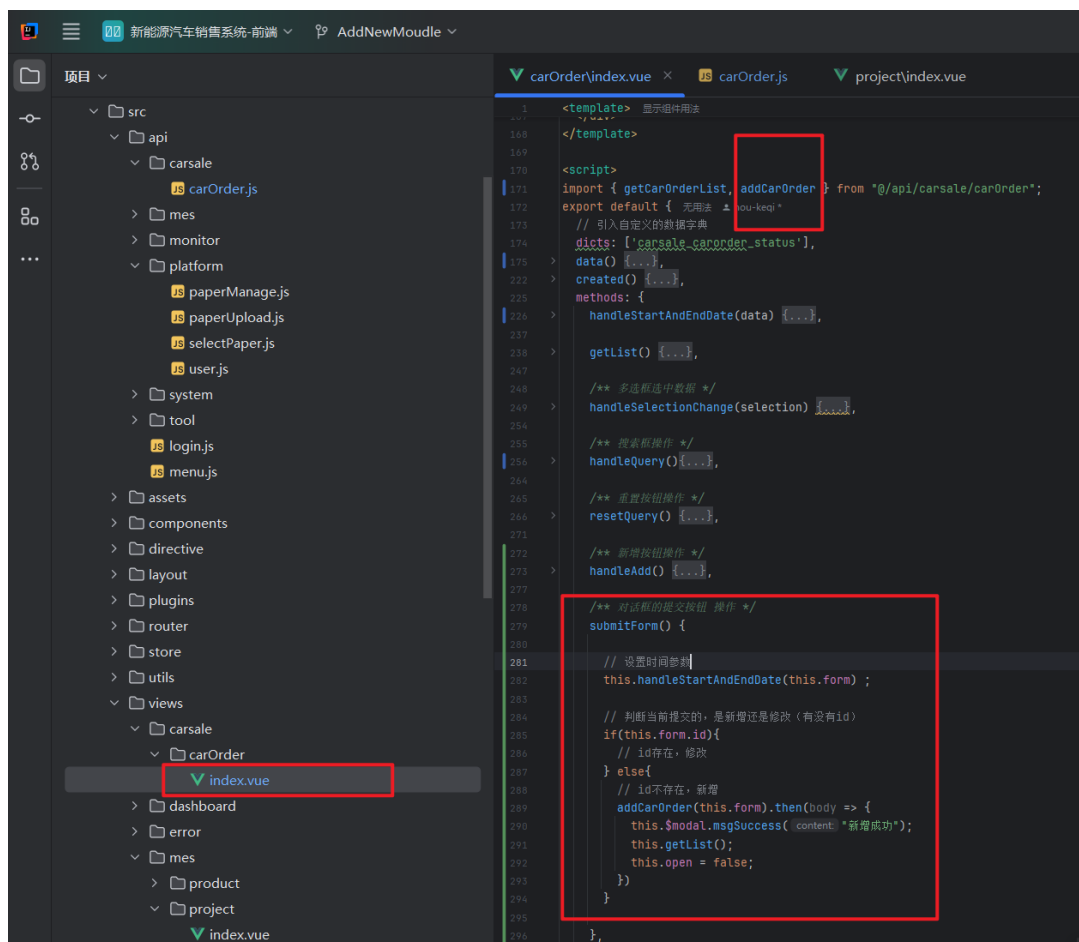
2. 补充“点击按钮”所调用的函数



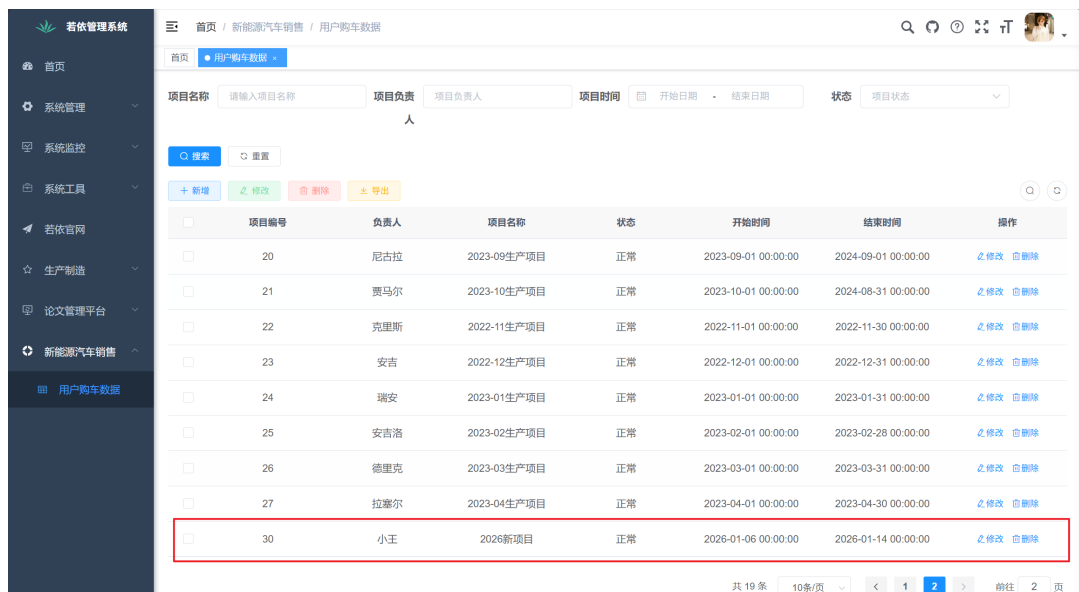
3. 新增功能所需要的api调用



别忘了要引入才能调用



其余各函数实现过程和细节，直接看代码会更快，这里就不一一贴图说明了



8. 其他的几个按钮，直接看代码吧，套路都一样的，这里有个坑我找了好久，在这记录一下

在 MyBatis 中，如果你的 `status` 是 `Integer` 类型且值为 `0`，MyBatis 会把 `0` 视为空字符串 `""`。导致这个 `if` 条件为 `false`，SQL 语句里就没拼上 `status` 的查询条件，结果就是查询了全表

```
26 <!-- 查询用户购车列表 -->
27
28 <select id="selectCarsaleCarOrderList" parameterType="CarsaleCarOrder" resultMap="CarsaleCarOrderMap">
29     select id, project_name, project_header, start_date, end_date, status, introduce from mes_project_maintenance
30     <where>
31         <if test="projectName != null and projectName != ''">            and project_name like concat('%', #{projectName}, '%')
32         <if test="projectHeader != null and projectHeader != ''">        and project_header like concat('%', #{projectHeader}, '%')
33         <if test="startDate != null and startDate != ''">                and start_date <= #{startDate}
34         <if test="endDate != null and endDate != ''">                    and end_date >= #{endDate}
35         <if test="status != null and status != ''">                      and status = #{status}
36     </where>
37 </select>
38
```